MACHINE LEARNING INTERVIEW PREPARATION

Classical NLP Foundations: Master Study Guide

Comprehensive Classical NLP Foundations & Systems

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Text Preprocessing, Vector Spaces, BM25, Embeddings, Language Models, RNN/LSTM/GRU, Drift Detection

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, Production Telemetry

Module 01: Introduction to NLP & Classical Tasks

1. Introduction & Intuition

The Core Bottleneck

Computers process structured, deterministic data (like integers and relational tables) in binary states. However, human language is unstructured, variable, and highly ambiguous. Early attempts to process language relied on manual regular expressions or context-free grammars (CFGs). These systems broke down on common language variations, misspellings, or contextual shifts. The core bottleneck was the lack of representations that could capture the multi-layered, fuzzy structure of human language.

High-Level Intuition

Think of language as a multi-layered code. To understand a message, you cannot just look at individual characters; you must analyze:

- How letters form words (Morphology).
- How words organize into grammatical sentences (Syntax).
- What those sentences literally denote (Semantics).
- How context and intent shape the message (Pragmatics).

NLP translates this multi-layered code into continuous numerical vectors that capture syntactic and semantic relations.

Walkthrough of a Sample String: `"Seattle's libraries are awesome! 🌟"`

In production systems, processing raw text requires translating unstructured strings into structured numerical matrices through a sequential pipeline:

Prediction → `Sentiment: POSITIVE (Confidence: 98.4%)`

Taxonomy of Classical Tasks

Classical NLP tasks are categorized into four major computational paradigms:

1. **Text Classification:** Assigning categorical labels to documents (e.g., spam detection).
2. **Sequence Tagging:** Assigning labels to individual words (e.g., POS tagging, NER).
3. **Syntactic Parsing:** Extracting structural syntax trees from sentences (e.g., dependency parsing).
4. **Text Generation:** Generating token sequences (e.g., translation, summarization).

---

2. Core Concepts & Mathematical Formulation

Rule-Based vs. Statistical NLP Representation

In early NLP, sequence matching used deterministic rule dictionaries. Modern systems replace these rules with statistical tag lookups.

Purpose & Intuition

Instead of manually writing rules for every word, statistical NLP models estimate the likelihood of a word belonging to a class based on counts from a training corpus. For example, in Part-of-Speech (POS) tagging, a word like `"run"` can be either a Noun (N) or a Verb (V). We use the corpus frequency to determine the most likely tag given the surrounding words.

Step-by-Step Probability Lookup Example

Let's resolve the Part-of-Speech tag for the word `"run"` in two different contexts.
Suppose we have a trained vocabulary dictionary containing:

- $P(\text{“run”}|\text{Verb}) = 0.80$
- $P(\text{“run”}|\text{Noun}) = 0.20$

If we encounter `"run"` in isolation, our model performs a direct probability lookup:

- $P(\text{Verb}|\text{“run”}) \propto 0.80$
- $P(\text{Noun}|\text{“run”}) \propto 0.20$
  The model predicts **Verb** because $0.80 > 0.20$.

Now, suppose we have contextual tag transition rules:

- If the previous word is an Article (e.g., `"the"`), the tag transition probability is $P(\text{Noun}|\text{Article}) = 0.90$ and $P(\text{Verb}|\text{Article}) = 0.10$.
- In the phrase `"the run"`, the model multiplies the word lookup probability by the transition likelihood:
  - **Score as Noun:** $P(\text{Noun}|\text{Article}) \times P(\text{“run”}|\text{Noun}) = 0.90 \times 0.20 = 0.18$
  - **Score as Verb:** $P(\text{Verb}|\text{Article}) \times P(\text{“run”}|\text{Verb}) = 0.10 \times 0.80 = 0.08$
    Comparing the scores ($0.18 > 0.08$), the model correctly tags `"run"` as a **Noun** in this context.

Tensor & Shape Tracking

- Input Sequence: `[N]` (list of word indices).
- Vocabulary Tag Probability Table: `[T, V]` (where T is the number of tags, V is vocabulary size).
- Output Sequence: `[N]` (predicted tag indices).

---

3. Implementation & Reference Code

Below is a Python regex classifier compared to a manual tag lookup.

```
def run_classification_demo():
    import re

    corpus = [
        "URGENT: Win a free cash prize now!",
        "Hello, are we still meeting for lunch today?",
    ]

    # Rule-Based Regex Tagger (deterministic pattern dictionary)
    spam_patterns = [r"urgent", r"prize", r"reward"]

    def rule_based_classify(text: str) -> str:
        normalized_text = text.lower()
        for pattern in spam_patterns:
            if re.search(pattern, normalized_text):
                return "SPAM"
        return "HAM"

    print("Rule-Based Predictions:")
    for idx, doc in enumerate(corpus):
        pred = rule_based_classify(doc)
        print(f"  Doc {idx+1}: {doc:<45} | Prediction: {pred}")

if __name__ == "__main__":
    run_classification_demo()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Representing text data features for categorical mapping.
- **Why Introduced over Legacy Approaches:** Statistical classification replaced manual regex rules because manual rules fail to scale or handle synonyms.
- **Key Failure Modes & Limitations:** Rule-based models are brittle; simple statistical probability models fail to capture syntactic context on out-of-vocabulary terms.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Regular expression matching runs in $O(M \times N)$ where M is pattern count and N is text length.
- **Space/Memory Footprint:** Space complexity scales as $O(|V|)$ for simple dictionary indexes.
- **Primary Bottleneck Type:** CPU-bound string search throughput.

3. Production & Scalability

- **Deployment Considerations:** Rule-based heuristics are frequently deployed as lightweight profanity filters or safety guards before hitting large-scale DL models.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Contrast syntactic parsing with semantic understanding. Why is semantic representation fundamentally harder?**
>
> Syntactic parsing maps grammatical linkages between tokens. Semantic understanding captures literal meaning. Semantics is harder because of polysemy (words with multiple meanings) and context dependency.

> **Question: What makes natural language unstructured, and what are the main dimensions of ambiguity?**
>
> Natural language is unstructured because it has no fixed record layout. The main dimensions of ambiguity are **lexical** (multiple word meanings), **syntactic** (multiple grammatical interpretations), and **semantic** (contextual intent).

Module 02: Text Preprocessing & Normalization

1. Introduction & Intuition

The Core Bottleneck

Machine learning models process static numerical vectors, not raw strings. To convert text into vectors, we must map words to a fixed vocabulary index. However, raw human text contains capitalization variance, punctuation, typos, and morphologic endings (e.g. `"runs"`, `"running"`, `"ran"`). If we assign a unique vector index to every variation, the vocabulary size ($|V|$) explodes. This increases model parameters (VRAM footprint) and splits statistical data signals, as the model must learn the meaning of `"ran"` and `"running"` independently. The bottleneck is compressing vocabulary variations without losing semantic context.

High-Level Intuition

Preprocessing is a lossy text compression pipeline. We convert raw text into a standardized, low-entropy canonical form by lowercase folding, stripping punctuation, filtering low-signal words (like `"the"` or `"is"`), and collapsing inflected words down to their root forms.

---

2. Core Concepts & Mathematical Formulation

Stemming vs. Lemmatization

To collapse inflected words down to roots, we use two paradigms:

1. **Stemming:** A heuristic, rule-based suffix truncation process. It acts on single words without grammatical context or dictionaries.
   - *The Porter Stemmer:* Uses structural rules (e.g. if word ends in `"-sses"`, map to `"-ss"`; if word ends in `"-ed"`, check syllable count and chop).
2. **Lemmatization:** A morphological analysis lookup process. It maps inflected words back to their dictionary root (*lemma*) by checking a dictionary (like WordNet) and utilizing the word's Part-of-Speech (POS) context.

Intuition & Practical Use

Stemming is a fast, lightweight regex tool used when processing massive corpora quickly (like search engine indexers). Lemmatization is slower but highly accurate, preferred when building grammatical understanding or semantic pipelines (like QA bots).

Unicode Normalization: NFC vs. NFD

Unicode characters can have equivalent visual representations but different binary code points:

- **NFD (Normal Form Decomposition):** Separates characters into base letters and accent marks.
  - *Example:* `"é"` is decomposed to base letter `"e"` (`\u0065`) and combining acute accent mark `"´"` (`\u0031`).
- **NFC (Normal Form Composition):** Combines letters and accents into a single code point.
  - *Example:* `"é"` is represented as a single character (`\u00E9`).

Normalizing all characters to a single form (typically NFC) prevents vocabularies from treating visually identical text as different words.

Hand Calculation on a Simple Example

Let's preprocess and compare stemming vs. lemmatization on the sequence:

$$X = \text{“wolves running quickly”}$$

- **Step 1: Stemming (Porter Algorithm Rules)**
  1. Word `"wolves"`:
     - Matches suffix rule: replace `"-ves"` with `"-f"`.
     - Output stem: `"wolv"` (not a valid dictionary word).
  2. Word `"running"`:
     - Matches suffix rule: replace `"-ning"` with `"-n"` (if consonant duplication is sliced).
     - Output stem: `"run"`.
  3. Word `"quickly"`:
     - Matches suffix rule: replace `"-ly"` with `"-li"`.
     - Output stem: `"quickli"`.
     - *Stemmed Output:* `["wolv", "run", "quickli"]`
- **Step 2: Lemmatization (WordNet Morphological Lookup)**
  1. Word `"wolves"`:
     - Identify POS: Noun.

    - WordNet mapping: matches plural form `"wolves"` → root singular `"wolf"`.
    - Output lemma: `"wolf"`.
2. Word `"running"`:
    - Identify POS: Verb.
    - WordNet mapping: matches progressive form `"running"` → base verb `"run"`.
    - Output lemma: `"run"`.
3. Word `"quickly"`:
    - Identify POS: Adverb.
    - WordNet mapping: no morphological root change.
    - Output lemma: `"quickly"`.
    - *Lemmatized Output:* `["wolf", "run", "quickly"]`

Notice that stemming produced non-words (`"wolv"`, `"quickli"`), while lemmatization yielded clean dictionary lemmas (`"wolf"`, `"quickly"`).

Tensor & Shape Tracking

- Input text: Raw string characters.
- Token Array: `[N_tokens]` on CPU during extraction passes.

---

3. Implementation & Reference Code

Below is a Python comparison of stemming and lemmatization on a sentence.

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import wordnet

# Download resources locally
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

def run_preprocessing_comparison():
    stemmer = PorterStemmer()
    lemmatizer = WordNetLemmatizer()

    words = ["studies", "studying", "went", "wolves", "running", "was"]
    pos_tags = {
        "studies": wordnet.NOUN,
        "studying": wordnet.VERB,
        "went": wordnet.VERB,
        "wolves": wordnet.NOUN,
```

```
        "running": wordnet.VERB,
        "was": wordnet.VERB
    }

    print(f"{'Original':<12} | {'Porter Stem':<15} | {'WordNet Lemma':<15}")
    print("-" * 48)
    for w in words:
        stemmed = stemmer.stem(w)
        tag = pos_tags.get(w, wordnet.NOUN)
        lemmed = lemmatizer.lemmatize(w, pos=tag)
        print(f"{w:<12} | {stemmed:<15} | {lemmed:<15}")

if __name__ == "__main__":
    run_preprocessing_comparison()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Word redundancy mitigation and structural vocabulary compression.
- **Why Introduced over Legacy Approaches:** Lemmatization is preferred over stemming when downstream semantic classification tasks require valid grammatical roots and word semantics.
- **Key Failure Modes & Limitations:** Lemmatizers fail when POS-tags are mispredicted. If `"saw"` is tagged as a Noun instead of a Verb, it fails to map to `"see"`.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Stemming runs in $O(L)$ where L is token length. Lemmatization runs in $O(L \log V_D)$ due to lookup tree structures inside WordNet.
- **Space/Memory Footprint:** Stemming requires zero VRAM. Lemmatization requires keeping the morphology database index loaded in RAM (approx. 10MB to 50MB).
- **Primary Bottleneck Type:** CPU-bound. Heavy string manipulation bottleneck during dataset token tokenization passes.

3. Production & Scalability

- **Deployment Considerations:** For web-scale indexing (e.g. search engines), stemming is integrated directly into the indexing phase. For modern neural nets (like LLMs), character-level subword tokenizers have largely replaced stemming and lemmatization, delegating root mapping to embedding parameter learning.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why is lemmatization computationally more expensive than stemming, and when is this trade-off justified?**
>
> Lemmatization requires checking grammatical context and loading a morphology lookup index (database) to resolve roots. The trade-off is justified when semantic accuracy and word validity are critical (e.g., question answering, semantic search), whereas stemming is better for speed-critical, retrieval-focused search indices.

> **Question: How does Unicode normalization prevent token fragmentation in subword models?**
>
> If Unicode characters like `"é"` exist as NFD (two code points `e` + accent) and NFC (one code point `é`), a subword model will tokenize them as separate tokens. Normalizing to a standard form NFC/NFKC ensures equivalent text is mapped to the same vocabulary indices, preventing vocabulary splits.

Module 03: Tokenization & Subword Algorithms

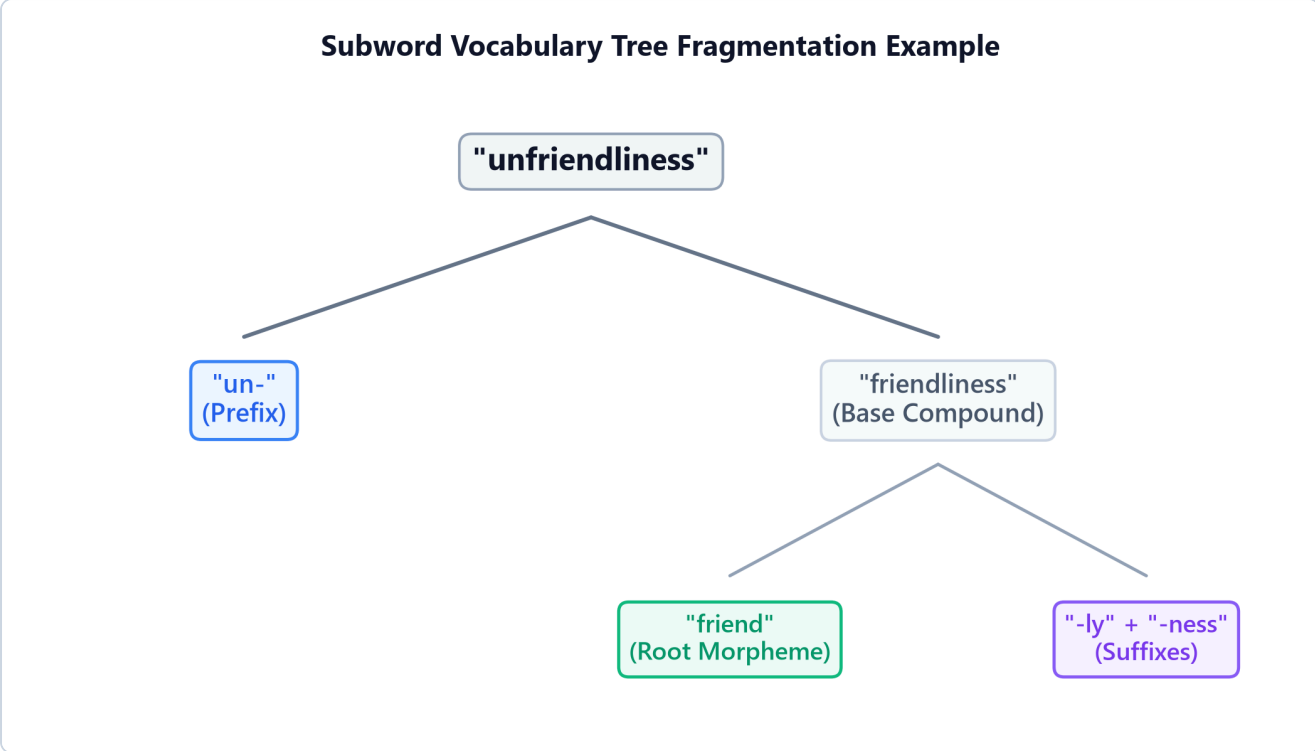
1. Introduction & Intuition

The Core Bottleneck

Language models require a mapping between textual units and continuous embedding vectors. Early models split text by spaces into words. This word-level tokenization creates a massive bottleneck: if a word is missing from the training dictionary (e.g. spelling mistakes, new names, slang), it gets mapped to an Out-of-Vocabulary (`<unk>`) token. This discards all semantic info. If we try to expand the vocabulary to include every possible word, the embedding layer parameters ($V \times d$) blow up, consuming gigabytes of VRAM. Character-level tokenization solves OOV but makes sequences extremely long (L), increasing attention complexity quadratically ($O(L^2)$) and diluting the semantic signal per token. The bottleneck is finding an optimal representations partition between sequence length L and vocabulary size V.

High-Level Intuition

Think of tokenization as structural compression. We want to represent text using a code. If the code only has single letters (characters), writing a message takes a long sequence. If the code has a unique symbol for every single word (word-level), we need an infinite catalog of symbols. Subword tokenization is the middle-ground: we start with letters, identify the most common letter combinations (like `"ing"`, `"est"`, `"trans"`), and assign them unique codes. If we meet a new word like `"unfriendliness"`, we split it into known subword blocks: `"un-"`, `"friend"`, and `"-liness"`. We preserve semantics without exploding the vocabulary.



---

2. Core Concepts & Mathematical Formulation

Subword Algorithms

Byte-Pair Encoding (BPE)

BPE starts by splitting all words in the training corpus into characters (adding a special end-of-word marker `_`). It counts the frequencies of all adjacent pairs (bigrams). The most frequent bigram is merged to form a new vocabulary token. This process repeats for a predefined number of merge iterations.

WordPiece

WordPiece is similar to BPE but differs in its merge selection criteria. Instead of picking the most frequent bigram, WordPiece selects the pair that maximizes the likelihood of the training data according to a unigram language model.

- **Intuition & Practical Use:** WordPiece measures how much more frequently two tokens appear together than expected by their individual frequencies, prioritizing meaningful morphological units over simple high-frequency character transitions.
- **Mathematical Scoring Formulation:**

$$\text{Score}(a, b) = \frac{\text{Count}(ab)}{\text{Count}(a) \times \text{Count}(b)}$$

SentencePiece

SentencePiece operates directly on raw byte streams without requiring pre-tokenization. It treats space as a standard character (represented by `_`), allowing it to build language-independent vocabularies without whitespace boundaries.

Hand Calculations

1. BPE Merge Step

Let's trace one merge step of the BPE algorithm.

- **Training Corpus:**
  1. `"l o w _"` (Frequency = 5)
  2. `"l o w e r _"` (Frequency = 2)
  3. `"n e w _"` (Frequency = 3)
- **Initial Vocabulary:** `{"e", "l", "n", "o", "r", "w", "_"}`
- **Step 1: Count adjacent bigrams in the corpus**
  - Bigram `('l', 'o')`: appears in `"low_"` (5 times) and `"lower_"` (2 times). Total = $5 + 2 = 7$.
  - Bigram `('o', 'w')`: appears in `"low_"` (5 times) and `"lower_"` (2 times). Total = $5 + 2 = 7$.
  - Bigram `('w', '_')`: appears in `"low_"` (5 times) and `"new_"` (3 times). Total = $5 + 3 = 8$.
  - Bigram `('w', 'e')`: appears in `"lower_"` (2 times). Total = 2.
  - Bigram `('e', 'r')`: appears in `"lower_"` (2 times). Total = 2.
  - Bigram `('e', 'w')`: appears in `"new_"` (3 times). Total = 3.
  - Bigram `('n', 'e')`: appears in `"new_"` (3 times). Total = 3.
- **Step 2: Select the maximum frequency bigram**
  - The maximum count is 8, corresponding to bigram `('w', '_')`.
- **Step 3: Merge the pair and update vocabulary**
  - New token created: `"w_"`.
  - Updated Vocabulary: `{"e", "l", "n", "o", "r", "w", "_", "w_"}`.
  - Updated Corpus:
    1. `"l o w_"` (Frequency = 5)

2. `"l o w e r _"` (Frequency = 2)
3. `"n e w_"` (Frequency = 3)

2. WordPiece Score Calculation

Let's calculate the WordPiece scores to decide which candidate pair to merge.

- Assume the corpus has:
  - Count(`"un"`) = 1000, Count(`"able"`) = 500, Count(`"unable"`) = 50.
  - Count(`"u"`) = 10000, Count(`"n"`) = 20000, Count(`"un"`) = 1000.
- **Candidate A (merge `"un"` + `"able"` into `"unable"`):**

$$\text{Score}(\text{“un”}, \text{“able”}) = \frac{\text{Count}(\text{“unable”})}{\text{Count}(\text{“un”}) \times \text{Count}(\text{“able”})} = \frac{50}{1000 \times 500} = \frac{50}{500000} = 0$$

- **Candidate B (merge `"u"` + `"n"` into `"un"`):**

$$\text{Score}(\text{“u”}, \text{“n”}) = \frac{\text{Count}(\text{“un”})}{\text{Count}(\text{“u”}) \times \text{Count}(\text{“n”})} = \frac{1000}{10000 \times 20000} = \frac{1000}{200000000} = 0.0$$

Since $\text{Score}(\text{“un”}, \text{“able”}) = 0.0001 > \text{Score}(\text{“u”}, \text{“n”}) = 0.000005$, the model merges `"un"` and `"able"` first, as they have higher mutual dependency.

Tensor & Shape Tracking

- Input Token ID matrix: `[B, L]` (where B is batch size, L is sequence length).
- Embedding lookup output: `[B, L, d]` (where d is embedding dimensions).

---

3. Implementation & Reference Code

Below is a self-contained BPE merge loop in Python.

```
import re
from collections import defaultdict

def run_bpe_demo():
    corpus = {
        "l o w _": 5,
        "l o w e r _": 2,
        "n e w e s t _": 6,
        "w i d e s t _": 3
    }
```

```
    vocab = set()
    for word in corpus.keys():
        vocab.update(word.split())

    def get_stats(corpus_dict):
        pairs = defaultdict(int)
        for word, freq in corpus_dict.items():
            symbols = word.split()
            for i in range(len(symbols) - 1):
                pairs[(symbols[i], symbols[i+1])] += freq
        return pairs

    def merge_vocab(pair, corpus_dict):
        new_corpus = {}
        bigram = re.escape(' '.join(pair))
        p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
        for word, freq in corpus_dict.items():
            new_word = p.sub(''.join(pair), word)
            new_corpus[new_word] = freq
        return new_corpus

    num_merges = 5
    for i in range(num_merges):
        pairs = get_stats(corpus)
        if not pairs:
            break
        best_pair = max(pairs, key=pairs.get)
        corpus = merge_vocab(best_pair, corpus)
        vocab.add(''.join(best_pair))
        print(f"Merge {i+1}: {best_pair} (Freq={pairs[best_pair]})")

    print("Final Vocabulary:", sorted(list(vocab)))

if __name__ == "__main__":
    run_bpe_demo()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The sequence length vs. vocabulary parameter trade-off, and Out-of-Vocabulary (OOV) containment.
- **Why Introduced over Legacy Approaches:** Subword tokenizers replaced word-level systems because they allow open-vocabulary representations (handling misspelled words, morphologic roots) at a fraction of the VRAM footprint required for full word lookups.
- **Key Failure Modes & Limitations:** Tokenizer-model mismatch (training a model on tokens generated by a different tokenizer vocabulary corrupts embedding layers completely).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Tokenization lookup runs in $O(N)$ where N is text character length, leveraging prefix trees (Tries) for fast matching.
- **Space/Memory Footprint:** The vocabulary mapping occupies $O(V)$ storage. Embedding lookup matrices scale as $V \times d$ parameters.
- **Primary Bottleneck Type:** Memory-bandwidth-bound during embedding lookup; CPU-bound during text preprocessing/tokenization loops.

3. Production & Scalability

- **Deployment Considerations:** Subword vocabularies are fixed prior to model pre-training. Changing the tokenizer vocab size later requires retraining the entire model's embedding weights from scratch.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: How do BPE and SentencePiece handle Out-of-Vocabulary (OOV) tokens at inference?**
>
> At inference, if a word is not directly present in the vocabulary, the subword tokenizer falls back to matching smaller character sequences down to individual byte tokens (SentencePiece byte-fallback). This guarantees that every text input is decomposed into a valid sequence of known token IDs, completely eliminating the `` tag.

> **Question: Detail the mathematical trade-off of choosing a small vocabulary size (e.g., 8k) vs. a large vocabulary size (e.g., 128k).**
>
> A small vocabulary size (8k) reduces embedding parameter count ($V \times d$), saving VRAM. However, it splits words into smaller subwords, increasing the sequence length L for the same sentence, which increases attention computational complexity quadratically ($O(L^2)$). A large vocabulary (128k) produces shorter sequence lengths but balloons the embedding layer VRAM footprint, which is a major constraint on consumer hardware.

Module 04: Vector Space Models (Bag-of-Words & TF-IDF)

1. Introduction & Intuition

The Core Bottleneck

Human documents vary in length, vocabulary choices, and grammatical style. To compare two text sequences algorithmically (e.g. for search retrieval or document clustering), we require a structured numerical coordinate space. Early keyword search pipelines matched exact strings, which failed to score partial overlaps or account for word frequencies. The bottleneck of early text retrieval was the lack of standard vector spaces where documents could be represented, normalized, and compared mathematically based on the semantic importance of their constituent terms.

High-Level Intuition

Think of document retrieval as locating coordinates in a multi-dimensional geometry space. If our vocabulary contains only two words—`"cat"` and `"dog"`—then every document can be represented as a coordinate vector in a 2D space. A document with three `"cat"` and one `"dog"` resides at coordinate $(3, 1)$. To calculate similarity between two documents, we calculate the angle (cosine) between their directional coordinate vectors. To prevent long documents with high raw word counts from dominating the space, we normalize vectors to unit length and penalize terms that appear everywhere (like `"the"`).

---

2. Core Concepts & Mathematical Formulation

Term Frequency (TF) & Inverse Document Frequency (IDF)

Intuition & Practical Use

TF measures how important a term is *inside* a document, while IDF measures how unique a term is *across* the entire collection. We multiply them to highlight terms that are highly descriptive of a specific document while filtering out universal words (like `"is"`, `"the"`) that do not help distinguish documents.

Mathematical Formulations

- **Smoothed IDF:**

$$\text{idf}_t = \log\left(\frac{1+N}{1+\text{df}_t}\right) + 1$$

  Where N is total documents and df_t is document frequency of term t.

- **TF-IDF Scoring:**

$$\text{tf-idf}_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

Cosine Similarity

Intuition & Practical Use

Since documents vary in length, comparing raw word count vectors would bias similarity scores toward long documents. Cosine similarity measures the angle between two document vectors, projecting them onto a unit sphere (L2 normalization) so that similarity represents semantic alignment rather than length.

Mathematical Formulation

$$\text{CosineSim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

Okapi BM25 Ranking Formulation

Okapi BM25 is a non-linear ranking function that scores document relevance to a query. It resolves two limits of TF-IDF:

1. **Term Frequency Saturation:** In TF-IDF, scores grow linearly with term frequency. BM25 scores saturate asymptotically. The parameter k_1 regulates this saturation rate.
2. **Document Length Normalization:** Longer documents naturally contain more words. BM25 scales the score based on document length relative to average document length avgdl, regulated by parameter b.

Mathematical Formulation

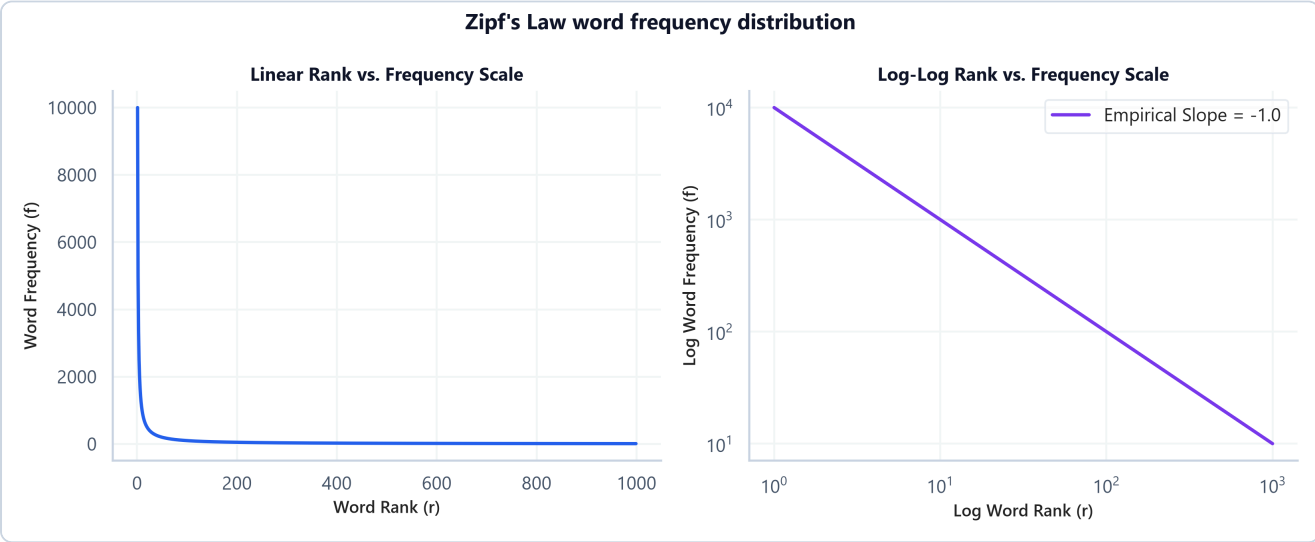
$$\text{Score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

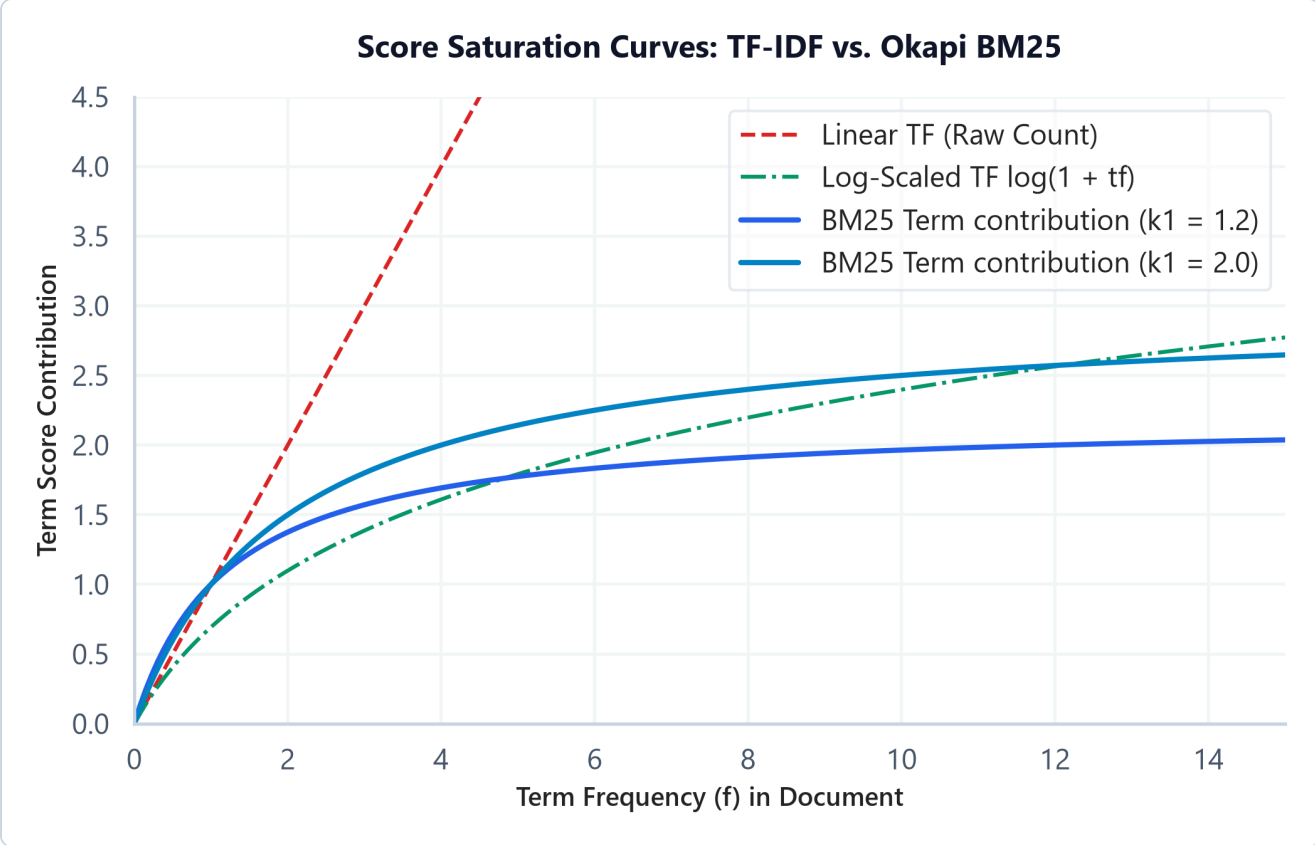
Where:

- $f(q_i, D)$ is the frequency of query term q_i in document D.
- $|D|$ is word length of document D.
- avgdl is the average document length.

Zipf's Law

Zipf's Law states that in a natural language corpus, the frequency of a word is inversely proportional to its frequency rank (a power-law decay where a tiny fraction of the vocabulary makes up the vast majority of occurrences). This justifies the log-scaling applied in TF and IDF to prevent common words from dominating representations.





Hand Calculation on a Simple Example

Let's calculate TF-IDF, Cosine Similarity, and BM25 relevance scores.

- **Corpus:**
  - Doc 1 (D_1): `"cat sat"` (length $|D_1| = 2$)
  - Doc 2 (D_2): `"cat running"` (length $|D_2| = 2$)
- **Query (Q):** `"cat sat"`
- **Parameters:** $N = 2$ documents, $\text{avgdl} = 2$. Let $k_1 = 1.2$, $b = 0.75$.
- **Vocabulary:** `{"cat", "sat", "running"}`
- **Step 1: Compute Document Frequencies (df) and smoothed IDF**

  1. Term `"cat"`: appears in D_1 and D_2. $\text{df} = 2$.

$$\text{IDF}(\text{“cat”}) = \log\left(\frac{1+2}{1+2}\right) + 1 = \log(1) + 1 = 1.0$$

2. Term `"sat"`: appears in D_1 only. $\text{df} = 1$.

$$\text{IDF}(\text{“sat”}) = \log\left(\frac{1+2}{1+1}\right) + 1 = \log(1.5) + 1 \approx 0.4055 + 1 = 1.4055$$

3. Term `"running"`: appears in D_2 only. $\text{df} = 1$.

$$\text{IDF}(\text{“running”}) = 1.4055$$

- **Step 2: Compute TF-IDF Vectors for Documents**
  - Doc 1 vector $\mathbf{u}$: `[cat=1, sat=1, running=0]`

$$\mathbf{u} = [1 \times 1.0, \quad 1 \times 1.4055, \quad 0] = [1.0, \quad 1.4055, \quad 0.0]$$

  - Doc 2 vector $\mathbf{v}$: `[cat=1, sat=0, running=1]`

$$\mathbf{v} = [1 \times 1.0, \quad 0, \quad 1 \times 1.4055] = [1.0, \quad 0.0, \quad 1.4055]$$

- **Step 3: Compute Cosine Similarity between Document 1 and Document 2**

1. Dot product $\mathbf{u} \cdot \mathbf{v}$:

$$\mathbf{u} \cdot \mathbf{v} = (1.0 \times 1.0) + (1.4055 \times 0.0) + (0.0 \times 1.4055) = 1.0$$

2. Vector Norms:

$$\|\mathbf{u}\|_2 = \sqrt{1.0^2 + 1.4055^2 + 0.0^2} = \sqrt{1 + 1.9754} = \sqrt{2.9754} \approx 1.7249$$

$$\|\mathbf{v}\|_2 = \sqrt{1.0^2 + 0.0^2 + 1.4055^2} \approx 1.7249$$

3. Cosine Similarity:

$$\text{CosineSim}(\mathbf{u}, \mathbf{v}) = \frac{1.0}{1.7249 \times 1.7249} = \frac{1.0}{2.9754} \approx 0.3361$$

The documents share a cosine similarity of 0.3361.

- **Step 4: Compute BM25 Query Score for Document 1 (D_1)**
  Query is `"cat sat"`. Length ratio $\frac{|D_1|}{\text{avgdl}} = 1.0$.

1. Score for `"cat"` ($f = 1$ in D_1):

$$\text{Score}_{\text{cat}} = \text{IDF}(\text{“cat”}) \times \frac{f \cdot (k_1 + 1)}{f + k_1 \cdot \left(1 - b + b \cdot \frac{|D_1|}{\text{avgdl}}\right)}$$

$$\text{Score}_{\text{cat}} = 1.0 \times \frac{1 \times 2.2}{1 + 1.2 \times (1 - 0.75 + 0.75 \times 1.0)} = \frac{2.2}{2.2} = 1.0$$

2. Score for `"sat"` ($f = 1$ in D_1):

$$\text{Score}_{\text{sat}} = 1.4055 \times \frac{1 \times 2.2}{1 + 1.2 \times 1.0} = 1.4055 \times 1.0 = 1.4055$$

3. Total BM25 score:

$$\text{Score}(D_1, Q) = 1.0 + 1.4055 = 2.4055$$

Tensor & Shape Tracking

- Document count matrix $\mathbf{X}_{\text{counts}}$: `[B, V]` (where B is batch size, V is vocabulary size).
- IDF vector: `[V]`.
- TF-IDF matrix: `[B, V]`.
- Similarity grid: `[B, B]`.

---

3. Implementation & Reference Code

Below is a NumPy implementation from scratch of TF-IDF, similarity scoring, and Okapi BM25 ranking.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

def run_vector_space_models():
    corpus = [
        "natural language processing models",
        "language modeling pipelines and systems",
        "production model serving systems"
    ]

    words = sorted(list(set(" ".join(corpus).split())))
    vocab = {word: idx for idx, word in enumerate(words)}
    V = len(vocab)
    B = len(corpus)

    bow_matrix = np.zeros((B, V))
    for i, doc in enumerate(corpus):
        for word in doc.split():
            bow_matrix[i, vocab[word]] += 1

    N = B
```

```
    df = np.sum(bow_matrix > 0, axis=0)
    idf = np.log((1 + N) / (1 + df)) + 1

    tfidf_raw = bow_matrix * idf
    norms = np.linalg.norm(tfidf_raw, axis=1, keepdims=True)
    tfidf_custom = tfidf_raw / (norms + 1e-15)

    vectorizer = TfidfVectorizer(smooth_idf=True, norm='l2')
    sklearn_tfidf = vectorizer.fit_transform(corpus).toarray()
    assert np.allclose(tfidf_custom, sklearn_tfidf, atol=1e-5)
    print("SUCCESS: Custom TF-IDF matrix matches Scikit-Learn exactly.")

    # Okapi BM25
    doc_lengths = np.array([len(doc.split()) for doc in corpus])
    avgdl = np.mean(doc_lengths)

    k1, b = 1.2, 0.75
    query = ["language", "models"]
    scores = np.zeros(B)

    for q_word in query:
        if q_word in vocab:
            col_idx = vocab[q_word]
            q_idf = idf[col_idx]
            q_tf = bow_matrix[:, col_idx]

            numerator = q_tf * (k1 + 1)
            denominator = q_tf + k1 * (1 - b + b * (doc_lengths / avgdl))
            scores += q_idf * (numerator / (denominator + 1e-15))

    print("\nBM25 Relevance Scores for query 'language models':")
    for doc_idx, score in enumerate(scores):
        print(f"  Doc {doc_idx+1}: {corpus[doc_idx]:<40} | Score: {score:.4f}")

if __name__ == "__main__":
    run_vector_space_models()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Document representation and relevance scaling for text retrieval.
- **Why Introduced over Legacy Approaches:** BM25 replaced raw TF-IDF in production systems because it bounds the impact of highly frequent terms (via term saturation asymptotes) and corrects document length discrepancies.
- **Key Failure Modes & Limitations:** Sparse keyword representations cannot capture semantic synonyms (e.g. searching `"automobile"` fails to retrieve documents containing only `"car"`).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Computing similarity matrix scores scales quadratically $O(B^2 \times V)$ for sparse matrix multiplications. For query scoring, BM25 scales linearly with query token count $O(L_{\text{query}} \times B)$.
- **Space/Memory Footprint:** The term-document matrix scales as $O(B \times V)$. Sparse representation structures (like CSR format) are used to drop zeros and save VRAM.
- **Primary Bottleneck Type:** Memory-bandwidth-bound during sparse dot-product index scans.

3. Production & Scalability

- **Deployment Considerations:** In production search engines (e.g. Elasticsearch, Vespa), BM25 serves as a high-speed "first-stage retriever" to narrow down millions of documents to a few hundred candidates, which are then passed to expensive dense cross-encoders.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why are Bag-of-Words and TF-IDF matrices sparse, and how does sparse representation affect memory footprint and matrix calculations?**
>
> They are sparse because a single document only contains a tiny subset of the global vocabulary V. Storing these as dense matrices wastes memory. We represent them as sparse data formats (like CSR - Compressed Sparse Row) that store only non-zero values and their indices. This drastically reduces the memory footprint and accelerates dot-products, as we skip calculations containing zeros.

> **Question: Explain Zipf's Law and how it justifies the log-scaling applied in TF and IDF.**
>
> Zipf's Law shows that word frequencies decay exponentially with rank. A few terms (like `"the"`) appear exponentially more often than rare words. If we scaled scores linearly with frequency, these terms would dominate. Log-scaling term frequencies compresses their impact, while IDF log-scaling dampens the effect of globally high-frequency terms.

> **Question: Explain why BM25 is preferred over TF-IDF for production retrieval. What do the parameters k_1 and b control?**
>
> BM25 is preferred because it prevents term frequency saturation and normalizes document length. The parameter k_1 controls the saturation limit: as a term repeats, its score contribution asymptotically plateaus rather than growing indefinitely. The parameter b controls length normalization: it penalizes long documents, preventing them from dominating rankings simply because they contain more words, while scaling the penalty based on average collection length.

Module 05: Distributed Representations (Word2Vec CBOW vs. Skip-gram)

1. Introduction & Intuition

The Core Bottleneck

In sparse vector space models (such as one-hot encoding or TF-IDF), every unique word in the vocabulary is mapped to an independent dimension. This architecture has a fatal flaw: word vectors are orthogonal. The dot-product between the one-hot vectors for `"cat"` and `"kitten"` is exactly 0. Sparse representations cannot capture semantic similarity or contextual relationships. Furthermore, storing these high-dimensional vectors ($V \approx 10^6$) balloons model parameters, leading to massive memory footprints and computational overhead. The bottleneck is the lack of compact, dense vector representations where semantic similarity maps directly to geometric proximity.

High-Level Intuition

Think of word embeddings as compressing language into a multi-dimensional semantic map. Instead of assigning a unique, independent dimension to every single word, we define a small, fixed coordinate space (e.g., $d = 300$ dimensions). We assign each dimension to represent a continuous semantic trait (like `"gender"`, `"plurality"`, or `"verbosity"`). Words that share traits project close to each other in this space. By moving from a high-dimensional sparse space to a low-dimensional dense space, we capture semantic concepts (e.g. `"king"` $-$ `"man"` $+$ `"woman"` $\approx$ `"queen"`).

2. Core Concepts & Mathematical Formulation

Word2Vec Architectures

Word2Vec is a framework for learning word embeddings using simple feedforward neural networks:

1. **Continuous Bag-of-Words (CBOW):** Predicts a target word w_t given its surrounding context words within a window C.
2. **Skip-Gram:** Predicts context words within window C given target word w_t.

Negative Sampling (SGNS)

Intuition & Practical Use

Computing a Softmax probability distribution over the entire vocabulary size V (which can be millions of words) at each update step is a severe bottleneck ($O(V)$). Negative sampling simplifies this into a binary classification problem: for each true context word (positive sample), we select k random words from the vocabulary (negative samples). The model is optimized to classify the true word as positive (1) and the noise words as negative (0). This reduces complexity from $O(V)$ to $O(k)$.

Mathematical Formulation

The objective function to minimize for a single target word w_I and positive context word w_O with k negative samples w_i is:

$$\mathcal{L}_{\text{NEG}} = -\log \sigma(\mathbf{v}'^{\top}_{w_O} \mathbf{v}_{w_I}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{v}'^{\top}_{w_i} \mathbf{v}_{w_I})$$

Where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid activation function.

- $\mathbf{v}_{w_I} \in \mathbb{R}^d$ is the input embedding of the target word.
- $\mathbf{v}'_{w_O} \in \mathbb{R}^d$ is the output embedding of the true context word.
- $\mathbf{v}'_{w_i} \in \mathbb{R}^d$ is the output embedding of the i-th negative sample.

Hand Calculation on a Simple Example

Let's compute the Negative Sampling Loss $\mathcal{L}_{\text{NEG}}$ for a single update step.

- **Dimensionality:** $d = 2$ dimensions.
- **Negative Samples count:** $k = 1$.
- **Target word input vector ($\mathbf{v}_{w_I}$):** $[1.0, 0.0]$
- **Positive context word output vector ($\mathbf{v}'_{w_O}$):** $[0.8, 0.6]$

- **Negative sample word output vector ($\mathbf{v}'_{w_{\text{neg}}}$):** $[0.5, -0.5]$
- **Step 1: Compute positive sample dot product and sigmoid score**

$$z_{\text{pos}} = \mathbf{v}'^{\top}_{w_O} \mathbf{v}_{w_I} = (0.8 \times 1.0) + (0.6 \times 0.0) = 0.8$$

$$\sigma(z_{\text{pos}}) = \frac{1}{1 + e^{-0.8}} = \frac{1}{1 + 0.4493} \approx 0.6901$$

$$\text{Positive Loss Term} = -\log(0.6901) \approx 0.3709$$

- **Step 2: Compute negative sample dot product and sigmoid score**

$$z_{\text{neg}} = \mathbf{v}'^{\top}_{w_{\text{neg}}} \mathbf{v}_{w_I} = (0.5 \times 1.0) + (-0.5 \times 0.0) = 0.5$$

$$\sigma(-z_{\text{neg}}) = \frac{1}{1 + e^{0.5}} = \frac{1}{1 + 1.6487} \approx 0.3775$$

$$\text{Negative Loss Term} = -\log(0.3775) \approx 0.9742$$

- **Step 3: Compute total Negative Sampling Loss**

$$\mathcal{L}_{\text{NEG}} = 0.3709 + 0.9742 = 1.3451$$

Our total loss value for this step is 1.3451. Backpropagation will compute gradients relative to this loss to push $\mathbf{v}_{w_I}$ closer to $\mathbf{v}'_{w_O}$ and further from $\mathbf{v}'_{w_{\text{neg}}}$.

Tensor & Shape Tracking

- Target index tensor: `[B]` (where B is batch size).
- Input Embeddings Matrix $\mathbf{W}_{\text{in}}$: `[V, d]`.
- Output Embeddings Matrix $\mathbf{W}_{\text{out}}$: `[V, d]`.
- Negative sample index tensor: `[B, k]`.
- Output Loss: `[]` (scalar float).

---

3. Implementation & Reference Code

Below is a PyTorch implementation of the Skip-Gram architecture with Negative Sampling.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SkipGramNegSampling(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int):
        super().__init__()
        self.in_embed = nn.Embedding(vocab_size, embed_dim)   # [V, d]
        self.out_embed = nn.Embedding(vocab_size, embed_dim)  # [V, d]

        torch.manual_seed(42)
        nn.init.xavier_uniform_(self.in_embed.weight)
        nn.init.xavier_uniform_(self.out_embed.weight)

      def forward(self, target: torch.Tensor, context: torch.Tensor, neg_samples:
torch.Tensor) -> torch.Tensor:
        # 1. Lookup Embeddings
        v_target = self.in_embed(target)        # [B, d]
        v_context = self.out_embed(context)      # [B, d]
        v_neg = self.out_embed(neg_samples)      # [B, k, d]

        # 2. Positive term score
        pos_score = torch.sum(v_target * v_context, dim=1) # [B]
        pos_loss = F.logsigmoid(pos_score)                 # [B]

        # 3. Negative term score
        v_target_expanded = v_target.unsqueeze(1)          # [B, 1, d]
        v_neg_transposed = v_neg.transpose(1, 2)           # [B, d, k]
         neg_score = torch.bmm(v_target_expanded, v_neg_transposed).squeeze(1) # [B,
k]
        neg_loss = torch.sum(F.logsigmoid(-neg_score), dim=1) # [B]

        return -torch.mean(pos_loss + neg_loss)

if __name__ == "__main__":
    B, V, d, k = 4, 1000, 32, 5
    model = SkipGramNegSampling(V, d)

    tgt = torch.randint(0, V, (B,))
    ctx = torch.randint(0, V, (B,))
    neg = torch.randint(0, V, (B, k))

    loss = model(tgt, ctx, neg)
    print(f"Skip-Gram Negative Sampling Model Loss: {loss.item():.4f}")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Representing semantic similarity geometrically using compact dense vector operations.
- **Why Introduced over Legacy Approaches:** Word2Vec replaced sparse models because dense embeddings map contextual similarity to cosine distance, solving semantic sparsity.
- **Key Failure Modes & Limitations:** Embeddings are static; they assign a single representation per word, failing to handle polysemy (e.g. `"bank"` has the same vector regardless of context).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Skip-gram with negative sampling scales as $O(B \times k \times d)$ calculations per update step, compared to $O(B \times V \times d)$ for full Softmax.
- **Space/Memory Footprint:** Requires storing two matrices: $2 \times V \times d$ parameters.
- **Primary Bottleneck Type:** Memory-bandwidth-bound during embedding weight index lookup and caching updates.

3. Production & Scalability

- **Deployment Considerations:** Word2Vec models are often pre-trained and saved as static lookup tables to speed up downstream models, avoiding real-time projection computation during serving.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Derive the computational complexity reduction of Skip-gram with negative sampling compared to Skip-gram with full Softmax.**
>
> With full Softmax, the model must calculate the score for the true context word and compute a partition normalization sum over the entire vocabulary size V, leading to $O(V \times d)$ operations. Negative sampling maps this to a binary classification problem: computing the dot-product for the target word, the positive context word, and k negative sample words. This reduces operations to $O(k \times d)$. For $V = 10^6$ and $k = 5$, this speeds up training by several orders of magnitude.

> **Question: Why does Skip-gram perform better on rare words compared to CBOW? Explain in terms of gradient updates.**
>
> In CBOW, context word embeddings are averaged into a single context representation to predict the target word. This averaging acts as a smoothing operation, diluting the signal of rare context words. In Skip-gram, each target word independently predicts context words, applying direct gradient updates to context embeddings. Rare target words update their context vectors directly, preserving semantic distinctness.

Module 06: Subword and Matrix-Based Embeddings (GloVe & FastText)

1. Introduction & Intuition

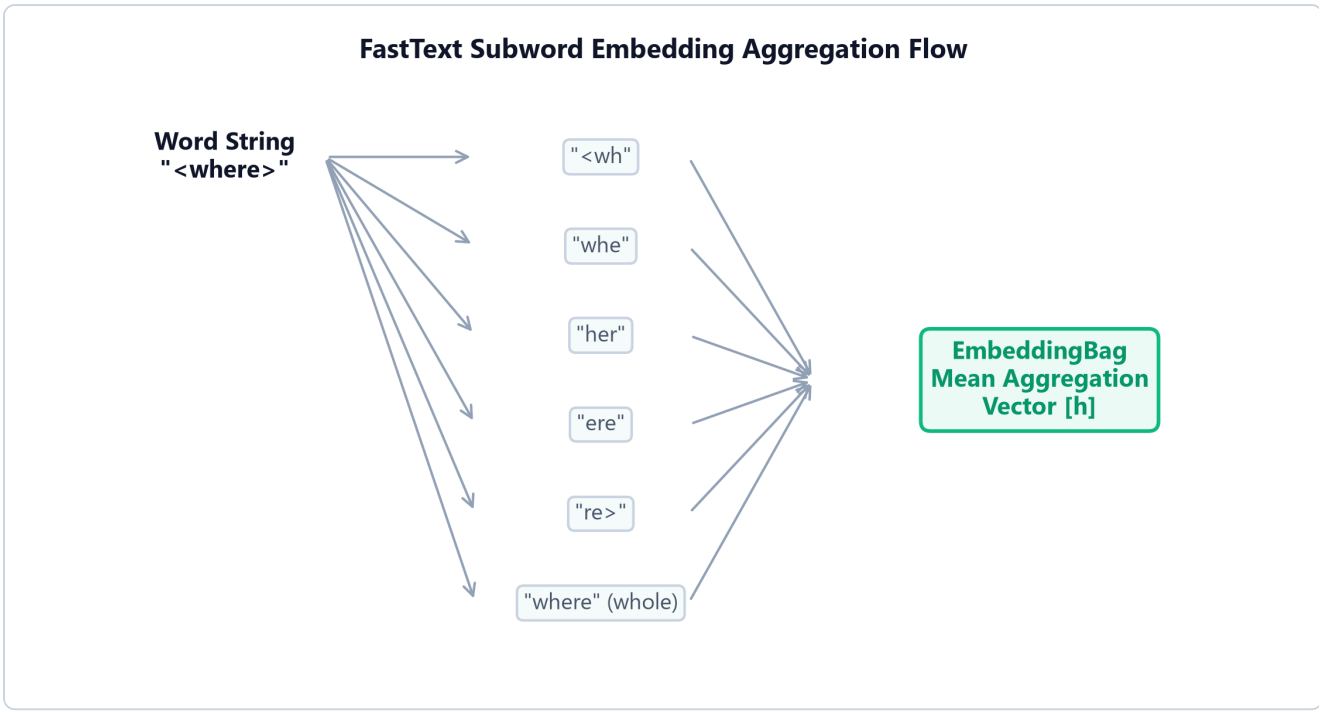
The Core Bottleneck

While Word2Vec embeddings capture local semantics, they suffer from two major limitations:

1. **Inefficient Context Window Scaling:** Word2Vec slides context windows over the corpus, repeating lookup calculations. It fails to utilize the global statistical ratios of word co-occurrences directly, wasting compute on redundant local contexts.
2. **The Out-of-Vocabulary (OOV) Wall:** Word2Vec learns representations at the word level. If a query word was not seen during training, or is a spelling variant (e.g. `"subword"` vs `"sub-word"`), the model cannot generate a vector representation, falling back to `<unk>`. The bottleneck is capturing global corpus-level co-occurrence statistics while retaining subword structural flexibility.

High-Level Intuition

Think of GloVe as building a global coordinate grid by factoring a giant co-occurrence spreadsheet. If we know how often all words appear next to each other globally, we can solve a constraint optimization problem that projects words into a coordinate grid where dot-products directly match the logarithm of their co-occurrence counts.
Think of FastText as breaking a word into a set of puzzle pieces (character n-grams). A word like `"<where>"` is represented as the sum of its subword segments: `"<wh"`, `"whe"`, `"her"`, `"ere"`, and `"re>"`. If we encounter an unseen word like `"<wherever>"`, we can synthesize its vector representation by summing the vectors of its constituent subword pieces.



---

2. Core Concepts & Mathematical Formulation

GloVe (Global Vectors)

Purpose & Intuition

Instead of sliding local context windows continuously, GloVe optimizes embeddings directly on global co-occurrence statistics. The core intuition is that the relationship between words is captured by the *ratio* of their co-occurrence probabilities with other probe words, rather than raw context counts.

- **Method:** GloVe solves a weighted least-squares regression problem over the entire co-occurrence matrix $\mathbf{X}$. The objective projects word vectors $\mathbf{w}_i$ and $\mathbf{w}_j$ such that their dot-product $\mathbf{w}_i^\top \mathbf{w}_j$ approximates the logarithm of their co-occurrence frequency $\log X_{ij}$.
- **Weighting Function $f(X_{ij})$:** To prevent highly frequent words (like `"the"`, `"and"`) from dominating the loss, GloVe dampens their influence using a scaling function:

$$f(X) = \begin{cases} \left(\frac{X}{x_{\max}}\right)^\alpha & \text{if } X < x_{\max} \\ 1.0 & \text{otherwise} \end{cases}$$

  Where $x_{\max} = 100$ and $\alpha = 0.75$ are standard production bounds.

FastText Subword Aggregation

Purpose & Intuition

To handle out-of-vocabulary (OOV) terms, FastText represents each word as the sum of its constituent character n-grams. During pre-training, the model learns embeddings for all subword fragments. At inference, if a word is unrecognized, FastText decomposes it and sums the vectors of its known subwords, preserving morphological signal.

Mathematical Formulation

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

Where:

- $\mathcal{G}_w$ is the set of all character n-grams representing word w.
- $\mathbf{z}_g \in \mathbb{R}^d$ is the embedding vector for n-gram g.

Hand Calculation on a Simple Example

1. GloVe Weighting Function

Let's compute the GloVe weight $f(X_{ij})$ for two word pairs.

- **Parameters:** $x_{\max} = 100$, $\alpha = 0.75$.
- **Case A (Rare co-occurrence):** Word i and j appear together 16 times ($X_{ij} = 16$).

$$f(16) = \left(\frac{16}{100}\right)^{0.75} = (0.16)^{0.75} \approx 0.2529$$

- **Case B (High-frequency co-occurrence):** Word i and j appear together 200 times ($X_{ij} = 200$). Since $200 \geq x_{\max}$:

$$f(200) = 1.0$$

> The weighting function bounds the contribution of very frequent word transitions, preventing them from dominating the global factorization loss.

2. FastText Subword Pooling

Let's synthesize the embedding representation for the word `"cat"`.

- **Dimension:** $d = 3$.

- **Character 3-grams of `"cat"`:** `{"<ca", "cat", "at>"}`.
- Assume the hashed bucket vectors for these n-grams are:
  - $\mathbf{z}_{\text{“<ca”}} = [0.1, \quad 0.2, \quad 0.3]$
  - $\mathbf{z}_{\text{“cat”}} = [0.0, \quad 0.5, \quad -0.1]$
  - $\mathbf{z}_{\text{“at>”}} = [0.5, \quad 0.2, \quad 0.4]$
- **Synthesized Word Vector calculation:**

$$\mathbf{v}_{\text{“cat”}} = \mathbf{z}_{\text{“<ca”}} + \mathbf{z}_{\text{“cat”}} + \mathbf{z}_{\text{“at>”}}$$

$$\mathbf{v}_{\text{“cat”}} = [0.1 + 0.0 + 0.5, \quad 0.2 + 0.5 + 0.2, \quad 0.3 - 0.1 + 0.4] = [0.6, \quad 0.9, \quad 0.6]$$

Tensor & Shape Tracking

- GloVe Co-occurrence Matrix $\mathbf{X}$: `[V, V]` (where V is vocabulary size).
- FastText subword n-gram indices: `[B, M]` (where B is batch size, M is n-grams per word).
- Output word embedding: `[B, d]`.

---

3. Implementation & Reference Code

Below is a PyTorch implementation of a FastText subword pooling lookup using `nn.EmbeddingBag`.

```
import torch
import torch.nn as nn

class FastTextEmbeddingBag(nn.Module):
    def __init__(self, num_buckets: int, embed_dim: int):
        super().__init__()
        # In EmbeddingBag, lookup and mean/sum pooling happen in a single kernel
                self.subword_embeddings = nn.EmbeddingBag(num_buckets, embed_dim,
mode='mean')

        torch.manual_seed(42)
        nn.init.xavier_uniform_(self.subword_embeddings.weight)

    def forward(self, subword_flat_indices: torch.Tensor, offsets: torch.Tensor) ->
torch.Tensor:
        # subword_flat_indices shape: [Total_Ngrams_In_Batch]
        # offsets shape: [B] (start index of each word in the flat array)
        return self.subword_embeddings(subword_flat_indices, offsets)

def simulate_fasttext_lookup():
    num_buckets = 1000
```

```
    embed_dim = 16
    model = FastTextEmbeddingBag(num_buckets, embed_dim)

    # 2 Words:
    # Word 1: index offsets 0-2 (indices [12, 45, 78])
    # Word 2: index offsets 3-6 (indices [99, 102, 105, 108])
    flat_indices = torch.tensor([12, 45, 78, 99, 102, 105, 108], dtype=torch.long)
    offsets = torch.tensor([0, 3], dtype=torch.long)

    with torch.no_grad():
        word_vectors = model(flat_indices, offsets)

    print("Output FastText Word Vectors Shape:", word_vectors.shape)
    print("Word Vector 1:\n", word_vectors[0])

if __name__ == "__main__":
    simulate_fasttext_lookup()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Global statistical co-occurrence representation efficiency, and Out-of-Vocabulary (OOV) reconstruction.
- **Why Introduced over Legacy Approaches:** FastText replaced Word2Vec in production pipelines because it resolves spelling mutations and handles rare words via n-gram subword pooling, which prevents out-of-vocabulary lookup failures.
- **Key Failure Modes & Limitations:** FastText hash collisions (different n-grams can hash to the same index, adding parameter noise).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** GloVe matrix construction scales as $O(\text{Corpus\_Size})$. FastText inference scales as $O(M \times d)$ operations per word, where M is the number of character n-grams.
- **Space/Memory Footprint:** FastText requires massive storage since it needs to store embeddings for all K_{buckets} n-grams (frequently $> 2\text{GB}$ on disk), unlike static word indices.
- **Primary Bottleneck Type:** Disk I/O and RAM loading latency during initialization; memory-bandwidth-bound during inference lookup.

3. Production & Scalability

- **Deployment Considerations:** Due to the large memory footprint of FastText hash tables, production serving pipelines often compress model weights using quantization (e.g. float16

conversion, product quantization) to reduce RAM footprints.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: What is the conceptual difference between Word2Vec's predictive window objective and GloVe's matrix-factorization global co-occurrence objective?**
>
> Word2Vec is a predictive model that slides context windows over the corpus, updating embeddings locally using stochastic gradient descent. It spends computations redundantly on frequent word pairs. GloVe is a count-based model that aggregates global counts over the entire corpus first, and then solves a matrix factorization problem. This directly models global co-occurrence statistics, resulting in faster convergence on large corpora.

> **Question: Detail the memory footprint penalty and inference speed trade-offs introduced by FastText subword storage systems during model loading.**
>
> Word2Vec only stores one vector per word, requiring $V \times d$ parameters. FastText must store vectors for both words and character n-grams. Because the number of possible subword n-grams is massive, FastText maps them to a large hash table (e.g., $K = 2\text{M}$ buckets), scaling memory parameters to $K \times d$. This increases model load times and RAM usage by several gigabytes, while adding tokenization overhead to parse words into n-grams at runtime.

Module 07: Statistical Language Models

1. Introduction & Intuition

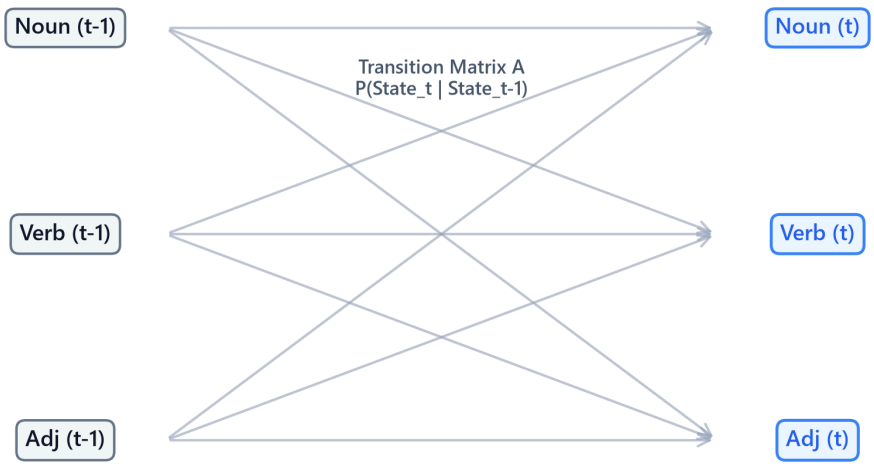
The Core Bottleneck

Human communication consists of predicting sequence continuations: given `"please turn off the"`, a human intuitively predicts `"lights"` over `"refrigerator"` or `"blue"`. Early NLP pipelines lacked a probabilistic framework to score the "naturalness" or likelihood of text sequences. To score candidate sentences in machine translation or speech recognition, models needed to evaluate joint token probabilities. However, calculating the joint probability of a sentence requires accounting for all possible histories, which creates a massive history dimensionality wall. The bottleneck is estimating sequence probability transitions accurately without running out of computer memory or encountering zero-probabilities on unseen histories.

High-Level Intuition

Think of sequence prediction as navigating a state transition highway. Instead of remembering the entire highway route you have driven since the start of your trip (the full history of words), you assume that your next turn depends only on the exit you are currently passing (the previous word or previous two words). This is the Markov assumption. By truncating the history window, we can estimate transition probabilities locally from corpus occurrence counts, making sequence scoring computationally feasible.

HMM Hidden State Transition Lattice Diagram



---

2. Core Concepts & Mathematical Formulation

The Chain Rule of Probability

To score a text sequence, we calculate its joint probability. The exact joint probability of a sequence of L words is factored sequentially using the chain rule:

$$P(w_1, w_2, \ldots, w_L) = \prod_{i=1}^{L} P(w_i | w_{<i})$$

The Markov Assumption & Bigrams

To make calculations tractable, N-gram models assume that the probability of a word depends only on the previous $N-1$ words. A **Bigram Model ($N = 2$)** looks back only at the single immediate preceding word:

$$P(w_1, \ldots, w_L) \approx \prod_{i=1}^{L} P(w_i | w_{i-1})$$

We estimate these transition probabilities from corpus counts using Maximum Likelihood Estimation (MLE):

$$P_{\text{MLE}}(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1} w_i)}{\text{Count}(w_{i-1})}$$

Laplace (Add-One) Smoothing

Purpose & Intuition

If a word combination (bigram) like `"natural refrigerator"` never appears in our training corpus, its MLE count is zero, collapsing the entire sequence joint probability to 0. Laplace smoothing prevents this score collapse by adding $+1$ to all possible transition combinations, reallocating a fraction of probability mass to unseen pairs.

Mathematical Formulation

$$P_{\text{Laplace}}(w_i|w_{i-1}) = \frac{\text{Count}(w_{i-1}w_i) + 1}{\text{Count}(w_{i-1}) + |V|}$$

Where $|V|$ is the size of the vocabulary.

Hidden Markov Model (HMM) sequence labeling

Purpose & Intuition

For sequence labeling (like Part-of-Speech tagging), we assume that word tokens are observed emissions generated by a sequence of hidden tags (states). HMMs use two matrices to find the most likely hidden tag path:

1. **Transition Matrix ($\mathbf{A}$):** Represents the probability of moving from one hidden tag to another (e.g. tag Verb following tag Noun).
2. **Emission Matrix ($\mathbf{B}$):** Represents the probability of a hidden tag generating a specific word (e.g. tag Verb emitting the word `"run"`).
   By multiplying transition and emission likelihoods, the model scores tag sequences.

Hand Calculation on a Simple Example

Let's compute Laplace smoothed bigram probabilities and evaluate the joint probability of a sequence.

- **Training Tokens:** `"language processing models language systems"`
  - Total tokens = 5.
  - Vocabulary (V): `{"language", "processing", "models", "systems"}`. Size $|V| = 4$.
- **Corpus Frequency Counts:**
  - Count(`"language"`) = 2.
  - Bigram `"language models"` occurs 1 time.
  - Bigram `"language systems"` occurs 0 times.

- **Step 1: Compute Laplace Smoothed step probabilities**

  1. Probability of `"models"` given `"language"`:

$$P_{\text{Laplace}}(\text{“models”}|\text{“language”}) = \frac{\text{Count}(\text{“language models”}) + 1}{\text{Count}(\text{“language”}) + |V|}$$

$$P_{\text{Laplace}}(\text{“models”}|\text{“language”}) = \frac{1+1}{2+4} = \frac{2}{6} \approx 0.3333$$

  2. Probability of `"systems"` given `"language"` (resolving the zero count):

$$P_{\text{Laplace}}(\text{“systems”}|\text{“language”}) = \frac{\text{Count}(\text{“language systems”}) + 1}{\text{Count}(\text{“language”}) + |V|}$$

$$P_{\text{Laplace}}(\text{“systems”}|\text{“language”}) = \frac{0+1}{2+4} = \frac{1}{6} \approx 0.1667$$

- **Step 2: Evaluate Joint Sequence Probability**
  Let's score the sequence: $W = [\text{“language”}, \text{“models”}]$.
  - Assume initial unigram probability for the starting word is:

$$P(\text{“language”}) = \frac{\text{Count}(\text{“language”}) + 1}{\text{Total Tokens} + |V|} = \frac{2+1}{5+4} = \frac{3}{9} \approx 0.3333$$

  - Joint Probability:

$$P(W) = P(\text{“language”}) \times P_{\text{Laplace}}(\text{“models”}|\text{“language”})$$

$$P(W) = 0.3333 \times 0.3333 \approx 0.1111$$

The joint sequence probability is 0.1111.

Tensor & Shape Tracking

- Bigram Count Matrix $\mathbf{C}_{\text{bigram}}$: `[V, V]`.
- Transition Probability Matrix $\mathbf{P}_{\text{transition}}$: `[V, V]`.

---

3. Implementation & Reference Code

Below is a Python implementation of bigram counts and sequence probability scoring.

```
import numpy as np

def run_bigram_language_model():
    corpus = "natural language processing language models are systems"
    tokens = corpus.split()

    vocab = sorted(list(set(tokens)))
    vocab_to_idx = {w: idx for idx, word in enumerate(vocab)}
    V = len(vocab)

    unigram_counts = np.zeros(V)
    bigram_counts = np.zeros((V, V))

    for t in tokens:
        unigram_counts[vocab_to_idx[t]] += 1

    for i in range(len(tokens) - 1):
        w1, w2 = tokens[i], tokens[i+1]
        idx1, idx2 = vocab_to_idx[w1], vocab_to_idx[w2]
        bigram_counts[idx1, idx2] += 1

    def get_bigram_prob(w_prev: str, w_curr: str) -> float:
        if w_prev not in vocab_to_idx or w_curr not in vocab_to_idx:
            return 1.0 / V
        idx_prev = vocab_to_idx[w_prev]
        idx_curr = vocab_to_idx[w_curr]
         return (bigram_counts[idx_prev, idx_curr] + 1) / (unigram_counts[idx_prev] +
V)

    test_seq = ["natural", "language", "models"]
    joint_prob = 1.0
    first_word_idx = vocab_to_idx[test_seq[0]] if test_seq[0] in vocab_to_idx else 0
    joint_prob *= (unigram_counts[first_word_idx] + 1) / (len(tokens) + V)

    for i in range(1, len(test_seq)):
        p_step = get_bigram_prob(test_seq[i-1], test_seq[i])
        joint_prob *= p_step

    print(f"Joint Probability of sequence {test_seq}: {joint_prob:.8f}")

if __name__ == "__main__":
    run_bigram_language_model()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Estimating sequence continuation probabilities and scoring generation candidates.
- **Why Introduced over Legacy Approaches:** N-gram models replaced full joint history estimation because the Markov assumption bounds history size, making sequence calculations feasible.
- **Key Failure Modes & Limitations:** History window truncation makes N-grams context-blind to dependencies longer than N tokens (e.g. a bigram model cannot check if a closing parenthesis matches an opening one 10 tokens prior).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Sentence scoring scales linearly with sequence length $O(L)$.
- **Space/Memory Footprint:** Space complexity scales exponentially with N-gram order: $O(|V|^N)$. A trigram model vocabulary index matrix grows as V^3.
- **Primary Bottleneck Type:** Memory capacity bounds (RAM footprint of N-gram tables).

3. Production & Scalability

- **Deployment Considerations:** Due to the exponential memory footprint of high-order N-grams, production systems rarely scale beyond trigram or 4-gram tables, instead using neural sequence models (RNNs/Transformers) to capture arbitrary history lengths in fixed embedding parameters.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does memory space complexity scale exponentially ($O(|V|^k)$) for statistical k-gram models, and how did this limitation motivate sequential deep learning architectures?**
>
> An N-gram model must allocate memory space for all potential token combinations: V values for unigrams, V^2 values for bigrams, and V^N values for N-grams. For a small vocabulary of 50k tokens, a trigram table requires allocating $50\text{k}^3 = 125 \times 10^{12}$ storage cells, which quickly exceeds computer RAM. Deep learning sequence models resolve this by compressing history into a fixed-size dense hidden state vector, scaling memory parameters linearly with layer depth rather than sequence history length.

> **Question: Explain backoff and interpolation smoothing techniques in statistical models.**
>
> **Backoff** checks if the higher-order N-gram (e.g. trigram) exists; if its count is zero, it "backs off" to estimate the probability using the lower-order bigram, scaling by a normalization weight. **Interpolation** computes a weighted linear combination of unigram, bigram, and trigram probabilities simultaneously: $P(w_i|w_{i-2}, w_{i-1}) = \lambda_1 P(w_i|w_{i-2}, w_{i-1}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_3 P(w_i)$, where $\sum \lambda_i = 1.0$.

Module 08: Classical NLP Evaluation Metrics

1. Introduction & Intuition

The Core Bottleneck

Building language pipelines requires standard, objective, and automated metrics to evaluate performance. In structured machine learning, we compare predictions against labels directly using accuracy or mean squared error. In NLP, however, output strings can have different lengths, word ordering, and synonyms. If a model translates a sentence as `"The cat sat on the rug"` and the reference is `"On the mat sat the cat"`, a simple token accuracy metric will score it as a failure. Evaluating text generation requires metrics that can handle semantic equivalence, sequence alignment variations, and probabilistic perplexity. The bottleneck is designing automated evaluation metrics that correlate well with human judgment without requiring expensive manual verification.

High-Level Intuition

Think of evaluation metrics as diagnostic magnifying glasses.
If we want to test how well a model has learned the language pattern probabilities, we measure how "surprised" it is when reading a test set of real text. If it assigns high probabilities to the real words, its surprise is low, which means its perplexity is low.
If we want to test translation quality, we count the overlap of word sequences (n-grams) between the model's translation and reference translations. If the model matches word pairs and triplets while preserving sentence length, its BLEU score is high.

---

2. Core Concepts & Mathematical Formulation

Perplexity (PPL)

Perplexity measures how well a language model predicts a sample.

- **Intuition & Practical Use:** Lower perplexity indicates the model is less surprised by the test words, meaning it has modeled the true probability distribution of the language accurately.

- **Mathematical Formulation:**

$$\text{PPL}(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_{<i})\right) = \exp(\text{Cross-Entropy Loss})$$

BLEU (Bilingual Evaluation Understudy)

BLEU measures the precision overlap of n-grams between candidate translation and reference translations. It scales the score using a **Brevity Penalty (BP)** to prevent models from outputting short sentences containing only a few highly confident words.

- **Intuition & Practical Use:** Evaluates machine translation quality automatically by penalizing word discrepancies and incorrect sentence lengths.
- **Mathematical Formulation:**

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{if } c \leq r \end{cases}$$

Where p_n is modified n-gram precision, $w_n = 1/N$, c is candidate length, and r is reference length.

Word Error Rate (WER)

WER is the standard metric for speech-to-text transcription. It measures the minimum edit distance (Levenshtein distance) at the word level, normalizing by reference length:

- **Mathematical Formulation:**

$$\text{WER} = \frac{S + D + I}{N}$$

Where S is substitutions, D is deletions, I is insertions, and N is total reference words.

Classification Metrics & Class Imbalance

In text classification (e.g. Spam detection), accuracy is an insufficient metric due to class imbalance. If 99% of emails are benign, a dummy classifier predicting "benign" constantly yields 99% accuracy while failing completely. We evaluate:

- **Precision:**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:**

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **F1-Score:** Harmonic mean of precision and recall:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Hand Calculations

1. BLEU Score with Brevity Penalty

Let's compute the BLEU-2 score for a candidate translation.

- **Reference (R):** `"the cat sat"` (length $r = 3$)
- **Candidate (C):** `"the cat"` (length $c = 2$)
- **Step 1: Compute modified n-gram precisions**
  1. **Unigram Precision (p_1):**
     - Candidate tokens: `["the", "cat"]`. Both appear in reference. Match count = 2.

$$p_1 = \frac{2}{2} = 1.0$$

  2. **Bigram Precision (p_2):**
     - Candidate bigrams: `["the cat"]`. Appears in reference. Match count = 1.

$$p_2 = \frac{1}{1} = 1.0$$

- **Step 2: Compute Brevity Penalty (BP)**
  Since candidate length $c = 2$ is less than reference length $r = 3$:

$$\text{BP} = \exp\left(1 - \frac{3}{2}\right) = \exp(-0.5) \approx 0.6065$$

- **Step 3: Compute BLEU-2 Score**
  Let weights $w_1 = 0.5$ and $w_2 = 0.5$:

$$\text{BLEU-2} = \text{BP} \times \exp\left(0.5 \log p_1 + 0.5 \log p_2\right)$$

$$\text{BLEU-2} = 0.6065 \times \exp\left(0.5 \log(1.0) + 0.5 \log(1.0)\right) = 0.6065 \times \exp(0) = 0.6065$$

The output candidate is penalized from 1.0 down to 0.6065 because it was too brief.

2. Word Error Rate (WER)

Let's compute WER for a speech-to-text hypothesis.

- **Reference (R):** `"the cat sat"` (length $N = 3$)
- **Hypothesis (H):** `"the cat was sitting"` (length = 4)
- **Step 1: Compute edit distance alignments**
  - `"the"` $\rightarrow$ `"the"` (Match)
  - `"cat"` $\rightarrow$ `"cat"` (Match)
  - `"sat"` $\rightarrow$ `"was"` (Substitution, $S = 1$)
  - Insert `"sitting"` (Insertion, $I = 1$). Deletions $D = 0$.
- **Step 2: Compute WER**

$$\text{WER} = \frac{S + D + I}{N} = \frac{1 + 0 + 1}{3} = \frac{2}{3} \approx 0.6667 \quad (66.7\%)$$

3. Classification Metrics

Let's compute metrics for a spam filter on a dataset of 100 emails.

- **Actual Labels:** 10 Spam, 90 Ham.
- **Classifier Predictions:**
  - True Positives (SPAM classified as SPAM): $\text{TP} = 8$.
  - False Negatives (SPAM classified as HAM): $\text{FN} = 2$.
  - False Positives (HAM classified as SPAM): $\text{FP} = 4$.
  - True Negatives (HAM classified as HAM): $\text{TN} = 86$.

- **Step 1: Compute Precision**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{8}{8+4} = \frac{8}{12} \approx 0.6667$$

- **Step 2: Compute Recall**

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{8}{8+2} = \frac{8}{10} = 0.8000$$

- **Step 3: Compute F1-Score**

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.6667 \times 0.8000}{0.6667 + 0.8000} = 2 \times \frac{0.5334}{1.4667} \approx 0.7273$$

The model achieves 66.7% precision, 80.0% recall, and a balanced F1-score of 0.7273.

Tensor & Shape Tracking

- Logits tensor: `[B, L, V]`.
- Cross-entropy loss output: `[]` (scalar float).
- Perplexity output: `[]` (scalar float).

---

3. Implementation & Reference Code

Below is a Python implementation of perplexity calculation from cross-entropy loss, and Levenshtein edit distance for WER computation.

```
import numpy as np
import torch
import torch.nn as nn

def run_evaluation_metrics():
    # 1. Compute Perplexity
    torch.manual_seed(42)
    B, L, V = 2, 4, 1000

    logits = torch.randn(B, L, V)
    targets = torch.randint(0, V, (B, L))

    loss_fn = nn.CrossEntropyLoss()
    loss = loss_fn(logits.view(-1, V), targets.view(-1))
    perplexity = torch.exp(loss)

    print(f"Cross-Entropy Loss: {loss.item():.4f}")
    print(f"Calculated Perplexity: {perplexity.item():.4f}")
```

```
    # 2. Compute WER
    def compute_wer(reference: str, hypothesis: str) -> float:
        r_words = reference.split()
        h_words = hypothesis.split()
        R, H = len(r_words), len(h_words)
        dp = np.zeros((R + 1, H + 1), dtype=int)

        for i in range(R + 1):
            dp[i, 0] = i
        for j in range(H + 1):
            dp[0, j] = j

        for i in range(1, R + 1):
            for j in range(1, H + 1):
                if r_words[i-1] == h_words[j-1]:
                    dp[i, j] = dp[i-1, j-1]
                else:
                    dp[i, j] = min(dp[i-1, j-1] + 1, dp[i-1, j] + 1, dp[i, j-1] + 1)

        return dp[R, H] / R

    ref = "the cat sat on the mat"
    hyp = "the cat was sitting on the mat"
    wer = compute_wer(ref, hyp)
    print(f"\nWER Score: {wer:.4f} ({wer*100:.1f}%)")

if __name__ == "__main__":
    run_evaluation_metrics()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Quantitative validation of generation alignment and model probabilities.
- **Why Introduced over Legacy Approaches:** Automating overlap scoring replaced slow, expensive human evaluations, allowing real-time model validation during training epochs.
- **Key Failure Modes & Limitations:** BLEU and ROUGE evaluate exact token overlap, completely penalizing synonyms (e.g. if candidate generates `"dog"` and reference is `"canine"`, overlap score is 0).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Perplexity computation scales as $O(B \times L \times V)$ due to vocabulary loss calculations. Edit distance for WER scales as $O(R \times H)$ where R and H are word counts.
- **Space/Memory Footprint:** DP grid for edit distance scales as $O(R \times H)$ memory.

- **Primary Bottleneck Type:** Compute-bound during logit-softmax evaluations over large vocabulary sizes.

3. Production & Scalability

- **Deployment Considerations:** For validating large-scale generation models (like LLMs), overlap metrics are often supplemented with semantic similarity lookups (like BERTScore) or LLM-based evaluations (G-Eval) to prevent synonym penalty errors.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Explain why perplexity is highly dependent on the vocabulary size ($|V|$), and why comparing perplexity values between two models with different tokenizers is mathematically invalid.**
>
> Perplexity is the exponent of the cross-entropy loss over a probability distribution of size $|V|$. A model with a smaller vocabulary size (e.g. 8k) naturally has a smaller pool of options to select from at each step compared to a model with a massive vocabulary (e.g. 128k). The cross-entropy loss of the smaller vocab model will be lower, yielding a lower perplexity simply due to vocab size, not model capability. Comparing perplexity is only valid when the vocabulary index size and tokenizer mappings are identical.

> **Question: What are the key architectural limitations of BLEU for evaluating translation semantic quality?**
>
> BLEU has three main limits: it penalizes synonyms because it matches exact string n-grams; it is insensitive to grammatical differences as long as n-gram counts match; and it does not capture sentence-level coherence or semantic intent, focusing only on local window overlaps.

Module 09: Recurrent Neural Networks (RNN, LSTM, GRU)

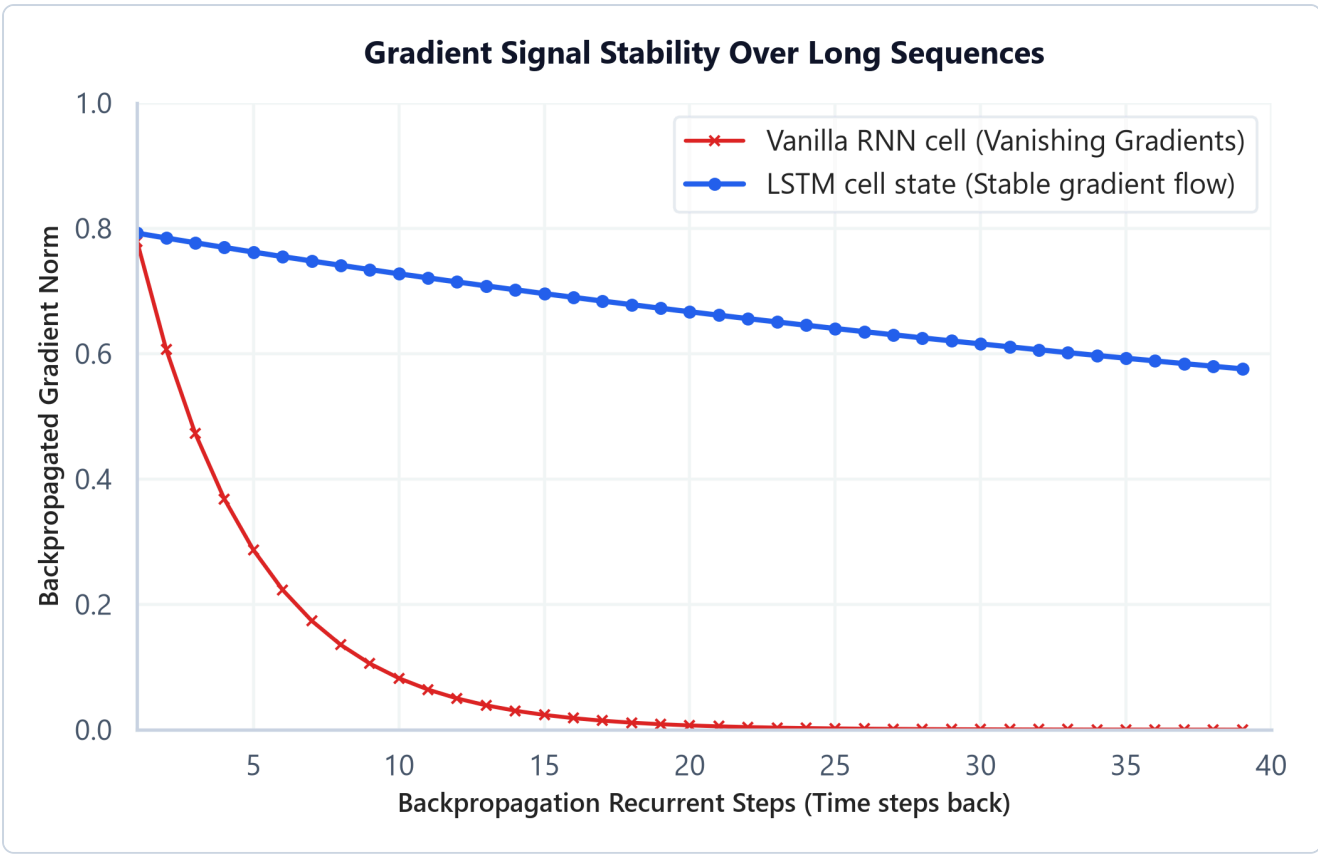
1. Introduction & Intuition

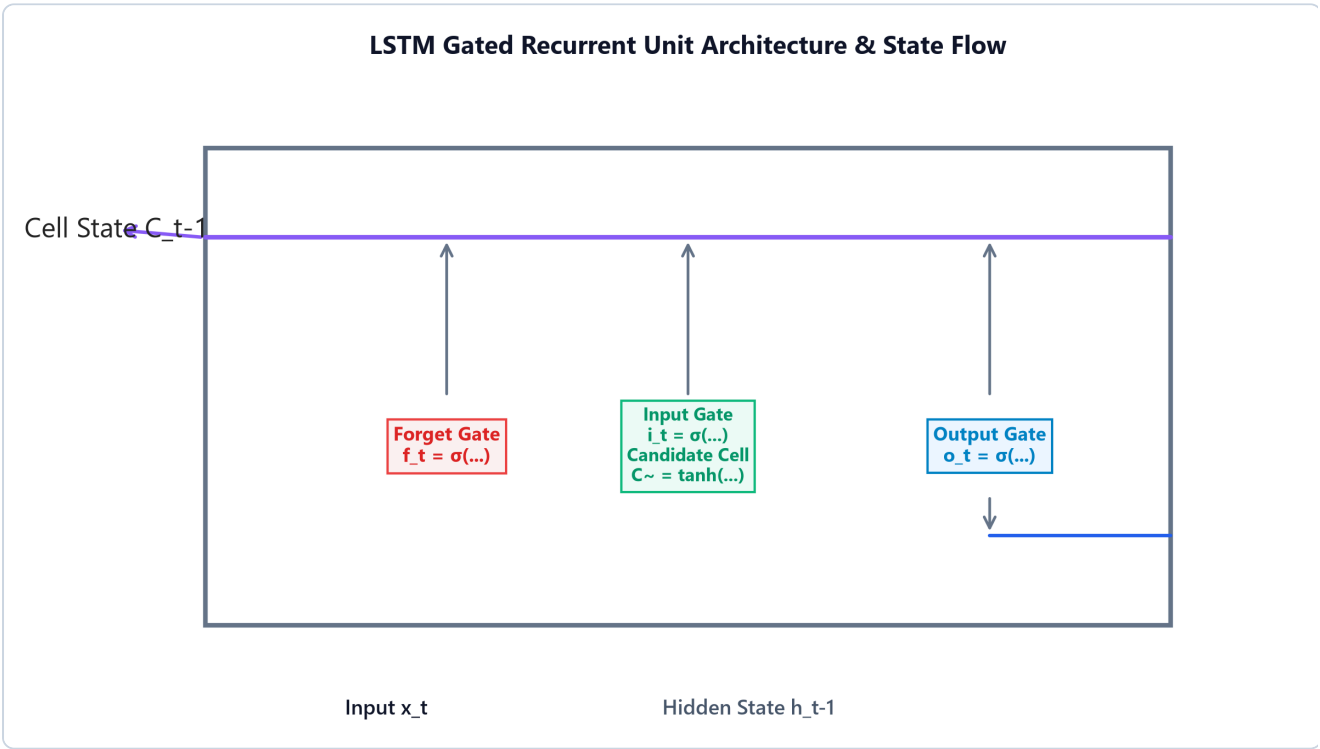
The Core Bottleneck

Standard feedforward neural networks (like MLPs) fail on natural language sequences. They require a fixed-size input vector (which cannot handle variable-length documents) and lack temporal memory (treating the word `"not"` in `"not good"` and `"good, not bad"` identically, ignoring position order). Early sequence modeling attempted to pad inputs, but this didn't capture temporal dependencies. The bottleneck of early deep NLP was the lack of sequential layers capable of keeping a running state history, sharing weights across variable-length time steps, and propagating gradients back through long temporal chains.

High-Level Intuition

Think of sequential reading as tracking a cognitive notepad (the hidden state). As you read a document word by word, you don't forget everything you read prior. Instead, you read the current word, merge it with what is currently on your cognitive notepad, erase low-value details, write down new high-value info, and update the notepad. This process repeats for every word. If the notepad has a linear carry path that isn't multiplied at each step, you can remember information from several sentences ago without losing the gradient signal.


Gradient Signal Stability Over Long Sequences
Vanilla RNN cell (Vanishing Gradients)
LSTM cell state (Stable gradient flow)
Backpropagated Gradient Norm
1.0
0.8
0.6
0.4
0.2
0.0
5
10
15
20
25
30
35
40
Backpropagation Recurrent Steps (Time steps back)


LSTM Gated Recurrent Unit Architecture & State Flow
Cell State C_t-1
Forget Gate
f_t = σ(...)
Input Gate
i_t = σ(...)
Candidate Cell
C~ = tanh(...)
Output Gate
o_t = σ(...)
Input x_t
Hidden State h_t-1

2. Core Concepts & Mathematical Formulation

The Recurrent Cell

A Recurrent Neural Network (RNN) processes input vectors $\mathbf{x}_1, \dots, \mathbf{x}_L$ sequentially, updating hidden state $\mathbf{h}_t$.

- **Intuition & Practical Use:** Replaces independent word lookups with a shared parameter recurrent cell that updates memory continuously over arbitrary sequence lengths.
- **Update Equation:**

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

Gradient Dynamics and the BPTT Bottleneck

Backpropagation Through Time (BPTT) & Gradient Decay

To train recurrent cells, we compute gradients by unrolling the model over sequence length L. The loss at the final step is backpropagated to the initial step using the chain rule. Because the hidden state weights $\mathbf{W}_{hh}$ are multiplied repeatedly (once for each time step), the gradient norm decays exponentially to zero if eigenvalues of $\mathbf{W}_{hh}$ are < 1.0 (vanishing gradient), or blows up to infinity if eigenvalues are > 1.0 (exploding gradient). This prevents standard RNNs from learning long-term dependencies.

The Gating Principle

To solve vanishing gradients, LSTMs introduce **Gates** (sigmoidal filters outputting values $\in [0, 1]$ to control memory writing, reading, and forgetting) and carry information linearly along an additive memory superhighway called the **Cell State** ($\mathbf{C}_t$).

LSTM Cell State Formulation

- **Forget Gate $\mathbf{f}_t$:** Decides what to drop from cell state (computed using sigmoid of context).
- **Input Gate $\mathbf{i}_t$ & Candidate Update $\tilde{\mathbf{C}}_t$:** Decides what new context to add to memory.
- **Cell State Update:** The core additive carriage equation that bypasses recurrent weight multiplications:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Hand Calculation on a Simple Example

1. Vanilla RNN Hidden State Update

Let's compute the updated hidden state $\mathbf{h}_t$ for a single recurrent cell.

- **Dimensions:** Hidden size $d_h = 1$, Input dimension $d_x = 1$.
- **Previous hidden state (h_{t-1}):** 0.5
- **Input token vector (x_t):** 1.0
- **Parameters:** Weight $W_{hh} = 0.8$, Weight $W_{xh} = 0.5$, bias $b_h = 0.0$.
- **Step 1: Compute pre-activation sum**

$$z_t = W_{hh}h_{t-1} + W_{xh}x_t + b_h$$

$$z_t = (0.8 \times 0.5) + (0.5 \times 1.0) + 0.0 = 0.4 + 0.5 = 0.9$$

- **Step 2: Apply non-linear tanh activation**

$$h_t = \tanh(z_t) = \tanh(0.9) = \frac{e^{0.9} - e^{-0.9}}{e^{0.9} + e^{-0.9}} \approx 0.7163$$

The updated hidden state is 0.7163.

2. LSTM Cell State Update

Let's trace how the LSTM additive cell state updates.

- **Previous Cell State (C_{t-1}):** 0.5
- **Gate Output Values:** Forget gate $f_t = 0.9$, Input gate $i_t = 0.8$, Candidate update $\tilde{C}_t = 0.5$.
- **Step 1: Compute updated cell state**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$C_t = (0.9 \times 0.5) + (0.8 \times 0.5) = 0.45 + 0.40 = 0.85$$

The new cell state is 0.85. Because the update is additive, gradients can propagate back through this path without multiplying by weight matrix factors.

Tensor & Shape Tracking

- Input Sequence Tensor $\mathbf{X}$: `[B, L, d_x]` where B is batch size, L sequence length, d_x input dimensions.
- Hidden state matrix $\mathbf{h}$: `[B, d_h]`.
- Bidirectional output tensor: `[B, L, 2 * d_h]`.

---

3. Implementation & Reference Code

Below is a PyTorch implementation of a Bidirectional LSTM.

```
import torch
import torch.nn as nn

class BidirLSTMClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int, num_classes:
int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=1,
            batch_first=True,
            bidirectional=True
        )
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        embedded = self.embedding(x) # [B, L, embed_dim]
        out, (h_n, c_n) = self.lstm(embedded)
        last_step = out[:, -1, :] # [B, hidden_dim * 2]
        return self.fc(last_step) # [B, num_classes]

def run_lstm_demo():
    B, L, V, embed_dim, hidden_dim, num_classes = 4, 15, 1000, 32, 64, 2
    model = BidirLSTMClassifier(V, embed_dim, hidden_dim, num_classes)
    inputs = torch.randint(0, V, (B, L))
    logits = model(inputs)
    print("Output Logits Shape:", logits.shape)

if __name__ == "__main__":
    run_lstm_demo()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Sequence state propagation, sequence classification, and variable-length text representation.
- **Why Introduced over Legacy Approaches:** LSTMs replaced vanilla RNNs because gating structures solve vanishing gradients, allowing models to learn long-range sequence context.
- **Key Failure Modes & Limitations:** LSTMs cannot model bidirectional context in their native forward state, and bidirectional models concatenate states, which increases vector parameter size.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Scales linearly with sequence length $O(L \times B)$ but cannot run in parallel.
- **Space/Memory Footprint:** VRAM parameters scale linearly with layer depth and hidden dimension: $O(4 \times (\text{embed\_dim} \times \text{hidden\_dim} + \text{hidden\_dim}^2))$.
- **Primary Bottleneck Type:** Memory-bandwidth-bound due to sequential matrix multiplications at each time step.

3. Production & Scalability

- **Deployment Considerations:** Due to the sequential dependency wall, LSTMs cannot leverage GPU parallelization along the sequence length dimension L during training, which makes them much slower to train than Transformers.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Explain mathematically why recurrent units are memory-bandwidth-bound during training and inference (the sequential dependency wall).**
>
> In an LSTM, the hidden state at step t ($\mathbf{h}_t$) cannot be computed until the state at step $t-1$ ($\mathbf{h}_{t-1}$) is completed. This serial constraint prevents parallelization across the sequence length dimension L. The GPU cannot run matrix operations for all words in parallel. Instead, it must read weights from High Bandwidth Memory (HBM) to SRAM, compute for one step, write back to HBM, and repeat L times. This makes the pipeline memory-bandwidth-bound rather than compute-bound.

> **Question: How does the additive cell state update mechanism in LSTMs ($\mathbf{C}_t$) resolve the vanishing gradient problem?**
>
> In vanilla RNNs, backpropagating gradients requires multiplying by weight matrix $\mathbf{W}_{hh}$ at each step, causing exponential decay. In LSTMs, the cell state updates using an additive equation: $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$. The gradient backpropagation path contains an additive term: $\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \mathbf{f}_t$. If the forget gate is open ($\mathbf{f}_t \approx 1.0$), the gradient propagates back through time steps linearly without decaying, creating an "error carousel".

Module 10: Production NLP Pipelines (Data Drift & Monitoring)

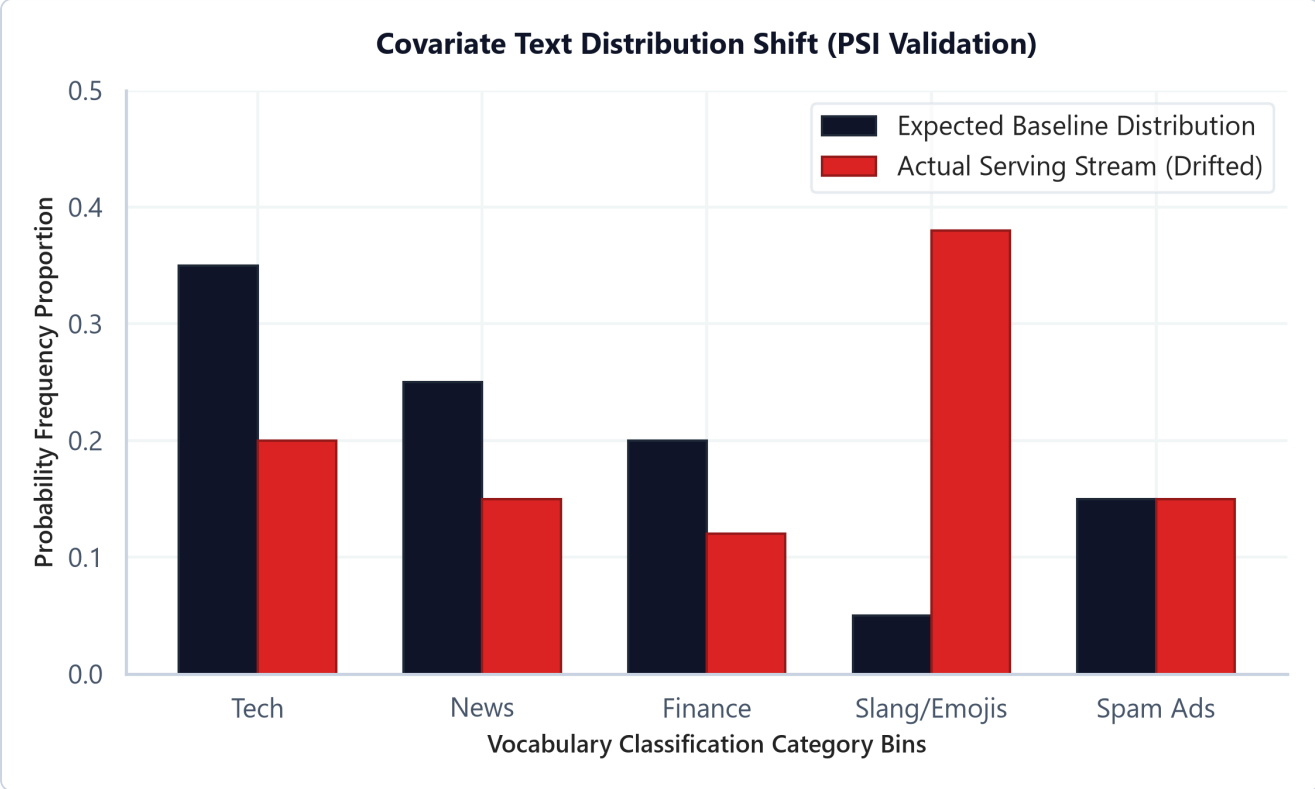
1. Introduction & Intuition

The Core Bottleneck

Deploying an NLP model to production is not a one-off task. Human language patterns change dynamically: users adopt new slang, shift topics due to global news, or alter their emojis. A text classification model (such as a spam filter or customer intent router) trained on historical logs will degrade in performance over time when serving live requests. This phenomenon is called data drift. The bottleneck of production NLP is detecting this distribution shift on unlabeled inference streams in real time, before model degradation affects users, and automating retraining paths under tight compute budgets.

High-Level Intuition

Think of data drift as a highway traffic shift. If your GPS maps the highway assuming 90% passenger cars and 10% trucks (your baseline expected distribution), but a new logistics hub shifts traffic to 50% trucks (actual serving distribution), the travel times and bottlenecks will change. In NLP, if a spam filter expects a baseline vocabulary distribution but starts receiving a stream of messages containing new spam emojis and slang (covariate shift), the classifier will misclassify inputs. We monitor this shift by binning incoming text frequencies and measuring the statistical distance (PSI) between actual and expected distributions.



---

2. Core Concepts & Mathematical Formulation

Data Drift Taxonomy

- **Covariate Shift:** The input feature distribution changes, but the conditional probability of the target label remains constant:

$$P(X_{\text{actual}}) \neq P(X_{\text{expected}}) \quad \text{while } P(Y|X) \text{ is unchanged}$$

  - *Example:* Users write spam with emojis (shift in X), but the definition of spam remains the same.

  - **Concept Drift:** The mapping between features and labels changes:

$$P(Y|X_{\text{actual}}) \neq P(Y|X_{\text{expected}}) \quad \text{while } P(X) \text{ is unchanged}$$

  - *Example:* Words that were benign (e.g. `"zoom"`) become associated with business meetings during a remote work transition.

Population Stability Index (PSI)

Intuition & Practical Use

PSI measures the extent of distribution shift between a reference dataset (Expected) and a target dataset (Actual) over B bins. In production, this calculates a single numeric value indicating if the model inputs have drifted enough to require retraining, without needing expensive human-labeled data.

- **PSI Interpretation Rules:**
  - $\text{PSI} < 0.1$: Stable; no significant distribution change.
  - $0.1 \leq \text{PSI} < 0.25$: Moderate shift; monitor the model and consider retraining.
  - $\text{PSI} \geq 0.25$: High shift; alert the pipeline and trigger retraining immediately.

Mathematical Formulation

$$\text{PSI} = \sum_{k=1}^{B} (\text{Actual}_k\% - \text{Expected}_k\%) \times \ln\left(\frac{\text{Actual}_k\%}{\text{Expected}_k\%}\right)$$

Hand Calculation on a Simple Example

Let's compute the Population Stability Index (PSI) for 2 vocabulary category bins: `["Tech", "Other"]`.

- **Expected (Baseline) distribution probabilities (p_k):**
  - $p_{\text{Tech}} = 0.80$
  - $p_{\text{Other}} = 0.20$
- **Actual (Serving) distribution probabilities (q_k):**
  - $q_{\text{Tech}} = 0.60$
  - $q_{\text{Other}} = 0.40$
- **Step 1: Compute contribution for the "Tech" bin**

  1. Difference:

$$\text{Diff}_{\text{Tech}} = q_{\text{Tech}} - p_{\text{Tech}} = 0.60 - 0.80 = -0.20$$

  2. Log Ratio:

$$\ln\left(\frac{q_{\text{Tech}}}{p_{\text{Tech}}}\right) = \ln\left(\frac{0.60}{0.80}\right) = \ln(0.75) \approx -0.2877$$

3. Contribution:

$$\text{Contr}_{\text{Tech}} = -0.20 \times -0.2877 = 0.0575$$

- **Step 2: Compute contribution for the "Other" bin**

1. Difference:

$$\text{Diff}_{\text{Other}} = q_{\text{Other}} - p_{\text{Other}} = 0.40 - 0.20 = 0.20$$

2. Log Ratio:

$$\ln\left(\frac{q_{\text{Other}}}{p_{\text{Other}}}\right) = \ln\left(\frac{0.40}{0.20}\right) = \ln(2.0) \approx 0.6931$$

3. Contribution:

$$\text{Contr}_{\text{Other}} = 0.20 \times 0.6931 = 0.1386$$

- **Step 3: Compute total PSI**

$$\text{PSI} = \text{Contr}_{\text{Tech}} + \text{Contr}_{\text{Other}} = 0.0575 + 0.1386 = 0.1961$$

- **Interpretation:** Since $0.10 \leq \text{PSI} = 0.1961 < 0.25$, we flag a **moderate shift**. The pipeline should monitor the incoming stream closely and prepare for retraining, but doesn't need to alert developers yet.

Tensor & Shape Tracking

- Expected baseline distribution: `[B]` where B is the number of bins.
- Actual counts vector: `[B]`.
- PSI output: Scalar float.

---

3. Implementation & Reference Code

Below is a Python telemetry monitor that calculates PSI.

```
import numpy as np

class TextDriftMonitor:
    def __init__(self, expected_dist: np.ndarray, categories: list):
        self.expected = expected_dist
        self.categories = categories
        self.B = len(categories)
          assert np.isclose(np.sum(self.expected), 1.0), "Expected distribution must
sum to 1.0"
```

```
    def compute_psi(self, actual_counts: np.ndarray) -> float:
        total_counts = np.sum(actual_counts)
        if total_counts == 0:
            return 0.0

        actual_dist = actual_counts / total_counts

        # Epsilon smoothing
        eps = 1e-5
        expected_smoothed = np.clip(self.expected, eps, 1.0 - eps)
        actual_smoothed = np.clip(actual_dist, eps, 1.0 - eps)

        expected_smoothed /= np.sum(expected_smoothed)
        actual_smoothed /= np.sum(actual_smoothed)

                    psi_value = np.sum((actual_smoothed - expected_smoothed) *
np.log(actual_smoothed / expected_smoothed))
        return psi_value

def run_drift_monitoring_demo():
    categories = ['Tech', 'News', 'Finance', 'Slang/Emojis', 'Spam Ads']
    expected_distribution = np.array([0.35, 0.25, 0.20, 0.05, 0.15])

    monitor = TextDriftMonitor(expected_distribution, categories)

    stable_counts = np.array([340, 260, 190, 60, 150])
    psi_stable = monitor.compute_psi(stable_counts)

    drifted_counts = np.array([200, 150, 120, 380, 150])
    psi_drifted = monitor.compute_psi(drifted_counts)

    print(f"Stable Window PSI:  {psi_stable:.4f} (Action: {'Retrain' if psi_stable >=
0.25 else 'Keep Model'})")
    print(f"Drifted Window PSI: {psi_drifted:.4f} (Action: {'Retrain' if psi_drifted
>= 0.25 else 'Keep Model'})")

if __name__ == "__main__":
    run_drift_monitoring_demo()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Detecting feature distribution shifts on unlabeled input streams to prevent silent model degradation.
- **Why Introduced over Legacy Approaches:** Statistical drift monitoring (PSI) replaced manual label checks because labeling production data is slow and expensive, while PSI can run in real-time

on raw inputs.

- **Key Failure Modes & Limitations:** Epsilon smoothing can skew PSI calculations on small sample windows; monitoring requires representative bin selections.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** PSI calculations run in $O(B)$ time, where B is the number of bins, adding negligible latency to inference pipelines.
- **Space/Memory Footprint:** Requires storing only reference distribution vectors: $O(B)$ parameters.
- **Primary Bottleneck Type:** I/O bound during telemetry logging and aggregation passes.

3. Production & Scalability

- **Deployment Considerations:** Telemetry services typically run asynchronously to prevent logging overhead from adding latency to the main model serving thread.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: What is covariate data drift in vocabulary distributions, and how can we detect it on an unlabeled serving stream in real time?**
>
> Covariate data drift in NLP occurs when the frequency distribution of input words changes over time (e.g. a sudden surge in specific hashtags or emojis). Because the serving stream is unlabeled, we cannot calculate model accuracy directly. Instead, we compute the Population Stability Index (PSI) or perform a Kolmogorov-Smirnov (KS) test comparing the word frequency distributions of the live stream against the training baseline. If the PSI exceeds 0.25, we flag significant drift.

> **Question: Detail the trade-offs of offline model batch retraining schedules vs. online real-time pipeline adjustments.**
>
> **Offline batch retraining** is stable and allows thorough evaluation, but it is slow and can lead to lag in adapting to sudden shifts. **Online real-time retraining** adapts instantly but is computationally expensive, prone to catastrophic forgetting, and risks learning noise or adversarial inputs, which can corrupt the model.

Module 11: Interview Questions & Answers

This module provides detailed answers for the 40 standard and 10 advanced bonus interview questions covering NLP foundations, mathematical derivations, debugging procedures, and production systems design.

---

1. Conceptual Questions (1-12)

Question 1: What is NLP, and what are the major NLP tasks?

Short Interview Answer (30–60 seconds)

Natural Language Processing (NLP) is the domain of artificial intelligence focused on enabling computers to understand, interpret, and generate human language. The major tasks include text classification (sentiment analysis, spam detection), sequence tagging (Named Entity Recognition, Part-of-Speech tagging), sequence-to-sequence translation, and question answering.

Key Interview Points

- Semantic analysis
- Sequence labeling (NER/POS)
- Text generation & translation

Production Perspective & Trade-offs

In production, different tasks require different latency limits. Classification runs under strict <50ms budgets using sparse algorithms, whereas sequence-to-sequence generation using autoregressive decoders incurs high computational cost and latency.

Follow-up Questions

- **Follow-up**: *What makes NER harder than text classification?* -> NER requires token-level context mapping and boundary extraction, whereas classification aggregates sentence embeddings.

Common Mistakes

- Confusing sequence labeling (NER) with sequence-to-sequence generation (translation).

Question 2: Explain a typical NLP pipeline from raw text to prediction.

Short Interview Answer (30–60 seconds)

The standard pipeline processes text through: raw input ingestion → text cleaning (normalization, regex) → tokenization (splitting into subwords) → text representation (mapping to sparse or dense embedding vectors) → model execution (running recurrent or self-attention layers) → prediction output (generating logits) → metric evaluation.

Key Interview Points

- Preprocessing & normalization
- Subword tokenization
- Vector embeddings
- Inference execution

Production Perspective & Trade-offs

Ensure preprocessing steps are identical between training and inference to prevent vocabulary mapping drift. Real-time inference pipelines often cache embedding weights to reduce database lookup overhead.

Follow-up Questions

- **Follow-up**: *Where does training-serving skew occur in this pipeline?* -> It usually happens when text normalization rules (like lowercasing or Unicode normalization) differ between training and serving code.

Common Mistakes

- Ignoring Unicode normalization, which leads to split character representations.

Question 3: What is the difference between stemming and lemmatization?

Short Interview Answer (30–60 seconds)

Stemming is a rule-based heuristic that truncates word endings (suffixes) to find a base form. Lemmatization uses morphological lookup tables and part-of-speech (POS) tags to resolve words to their canonical dictionary base form (lemma).

Key Interview Points

- Heuristic truncation (Stemming)

- Morphological dictionary lookup (Lemmatization)
- POS tagging dependency

Production Perspective & Trade-offs

- **Stemming**: Extremely fast, low memory usage, but can produce non-words (e.g. `"studies"` → `"studi"`).
- **Lemmatization**: Grammatically accurate, but has high latency due to POS tagging and dictionary lookups.

Follow-up Questions

- **Follow-up**: *Which is preferred for web search indexing?* -> Lemmatization is preferred because it maps words accurately to real dictionary terms, improving search query recall.

Common Mistakes

- Assuming stemming is always superior because of its lower execution latency.

Question 4: Why is tokenization necessary?

Short Interview Answer (30–60 seconds)

Computers cannot process unstructured strings directly. Tokenization decomposes text streams into discrete lexical chunks (words, subwords, or characters) that can be mapped to unique indices in a model's vocabulary table.

Key Interview Points

- Lexical splitting
- Vocabulary mapping
- Index lookup tables

Production Perspective & Trade-offs

Choosing the tokenization boundary directly impacts model parameter count. A very large vocabulary increases the size of the final projection layer, whereas a small character-level vocabulary increases input sequence lengths, raising attention computation costs.

Follow-up Questions

- **Follow-up**: *Can we build an NLP system without a tokenizer?* -> Yes, by processing raw bytes directly (e.g. Byte-level models), but this increases training times and sequence lengths.

Common Mistakes

- Treating tokenization as a simple string split on whitespace, which fails to handle punctuation and clitics (like `"don't"`).

Question 5: Compare character, word, and subword tokenization.

Short Interview Answer (30–60 seconds)

Character tokenization splits text into letters, resulting in small vocabularies but long sequence arrays. Word tokenization splits on word boundaries, keeping sequence lengths short but creating massive vocabularies with high out-of-vocabulary (OOV) rates. Subword tokenization (BPE/WordPiece) strikes a balance by splitting common roots while decomposing rare words into sub-components.

Key Interview Points

- Vocabulary size vs. Sequence length
- Out-of-Vocabulary (OOV) rates
- Computational balance

Production Perspective & Trade-offs

Subword tokenizers (like WordPiece) are standard in production LLMs because they keep vocabulary tables compact ($32\text{k}–256\text{k}$) while preventing OOV errors during user queries.

Follow-up Questions

- **Follow-up**: *Which tokenizer type is most robust against typos?* -> Character-level or subword tokenizers, as they can decompose misspelled words into known character sequences.

Common Mistakes

- Believing character-level tokenization is more computationally efficient (it actually increases sequence length, raising attention computational cost quadratically).

Question 6: Why did modern NLP move from word tokenization to subword tokenization?

Short Interview Answer (30–60 seconds)

Word-level tokenization struggles with vocabulary explosion ($|V| > 1,000,000$) and high OOV rates. Subword tokenization represents text using a smaller vocabulary of root prefixes and suffixes, allowing

models to process new or misspelled words without crashing or generating `<unk>` tokens.

Key Interview Points

- Vocabulary size limits
- Out-of-Vocabulary (OOV) mitigation
- Subword decomposition

Production Perspective & Trade-offs

Subword vocabularies fit easily into GPU memory, freeing up VRAM for longer sequence lengths and larger model hidden dimensions.

Follow-up Questions

- **Follow-up**: *How does BPE handle numerical data?* -> It often splits numbers into individual digits or byte sequences, preventing the model from needing a separate token for every integer.

Common Mistakes

- Believing subwords are manually defined by linguists (they are learned statistically from training data).

Question 7: What are Out-of-Vocabulary (OOV) words, and how can they be handled?

Short Interview Answer (30–60 seconds)

OOV words are tokens encountered during inference that were not present in the training vocabulary. They can be handled by mapping them to a fallback `<unk>` token, utilizing subword tokenizers (BPE) to decompose them, or using character-level models like FastText.

Key Interview Points

- Unknown tokens
- Byte-fallback vocabularies
- FastText n-gram recovery

Production Perspective & Trade-offs

Using `<unk>` degrades model accuracy because the model loses the semantic content of the unknown token. FastText handles this by generating vectors from subword n-grams.

Follow-up Questions

- **Follow-up**: *Why does BERT not return OOV errors?* -> BERT uses WordPiece tokenization, which decomposes unknown words down to individual characters if needed.

Common Mistakes

- Believing that increasing training vocabulary size to include every possible word completely resolves OOV issues.

Question 8: Explain the Distributional Hypothesis.

Short Interview Answer (30–60 seconds)

The Distributional Hypothesis states that words that occur in similar contexts share semantic meaning. This forms the foundation for dense word representations: models learn embeddings by predicting words based on their neighbors.

Key Interview Points

- Contextual semantics
- Word co-occurrences
- Vector projection spaces

Production Perspective & Trade-offs

This hypothesis allows models to learn representations from unstructured text, eliminating the need for expensive manual semantic labeling.

Follow-up Questions

- **Follow-up**: *What is a limitation of this hypothesis?* -> Antonyms (like `"hot"` and `"cold"`) often appear in identical contexts, leading to similar vector representations despite having opposite meanings.

Common Mistakes

- Assuming the hypothesis requires dictionary-defined semantic tags to compute vector similarities.

Question 9: Why do sparse text representations fail to capture semantic similarity?

Short Interview Answer (30–60 seconds)

Sparse representations (One-Hot, Bag of Words) treat each word as an independent, orthogonal dimension. Consequently, the dot product between different words (e.g. `"cat"` and `"feline"`) is 0, capturing zero semantic overlap.

Key Interview Points

- Vector orthogonality
- Sparse dimensionality
- Zero-product overlap

Production Perspective & Trade-offs

Sparse representations scale with vocabulary size ($|V|$), leading to high memory footprint and data sparsity during downstream training.

Follow-up Questions

- **Follow-up**: *How does Cosine Similarity help evaluate sparse vectors?* -> It measures overlap on shared coordinates, but fails if documents use synonyms.

Common Mistakes

- Assuming TF-IDF captures word similarities (it only matches exact token strings).

Question 10: Compare static embeddings and contextual embeddings.

Short Interview Answer (30–60 seconds)

Static embeddings (Word2Vec, GloVe) assign a single fixed vector to each word, regardless of context. Contextual embeddings (BERT, GPT) generate dynamic vectors for each token based on the surrounding sentence context.

Key Interview Points

- Polysemy resolution
- Static vocabulary tables
- Dynamic encoder projections

Production Perspective & Trade-offs

- **Static**: Fast, constant $O(1)$ memory lookup, but cannot resolve word meanings dynamically.
- **Contextual**: High computation cost (requires transformer forward passes), but resolves word context (e.g., `"bank"` in `"river bank"` vs. `"money bank"`).

Follow-up Questions

- **Follow-up**: *Can static embeddings resolve part-of-speech tags?* -> No, because the word vector remains identical regardless of whether it is used as a noun or a verb.

Common Mistakes

- Assuming static embeddings are obsolete (they remain useful for low-latency similarity searches).

Question 11: Why were RNNs introduced after Bag-of-Words and TF-IDF?

Short Interview Answer (30–60 seconds)

Bag-of-Words and TF-IDF discard word order and syntax. RNNs process tokens sequentially, updating a hidden state to capture temporal dependencies and word order.

Key Interview Points

- Sequence order preservation
- Variable length handling
- Hidden state memory

Production Perspective & Trade-offs

RNNs process text sequentially, creating a training bottleneck that prevents parallel GPU execution.

Follow-up Questions

- **Follow-up**: *How does RNN state size impact VRAM usage?* -> Larger hidden states require storing larger intermediate vectors during Backpropagation Through Time (BPTT).

Common Mistakes

- Believing RNNs can process tokens in parallel like Transformers.

Question 12: Why were Transformers able to replace RNNs?

Short Interview Answer (30–60 seconds)

Transformers replaced RNNs by utilizing self-attention, which processes all tokens in parallel and reduces token-to-token path lengths to $O(1)$. This allows for faster training speeds and better scaling on long sequences.

Key Interview Points

- Parallel training path
- Long-range dependencies ($O(1)$)
- Self-attention projection

Production Perspective & Trade-offs

Transformers scale training efficiently but require quadratic $O(L^2)$ memory during inference due to the attention matrix computation.

Follow-up Questions

- **Follow-up**: *Why do Transformers need positional encodings?* -> Self-attention is permutation-invariant; without positional encodings, it would treat sentences as unordered bags of words.

Common Mistakes

- Assuming Transformers require less memory than RNNs (their attention matrix is memory-intensive).

---

2. Mathematical Questions (13-22)

Question 13: Derive the TF-IDF equation and compute TF-IDF scores for a small corpus.

Short Interview Answer (30–60 seconds)

TF-IDF (Term Frequency-Inverse Document Frequency) measures a token's information density relative to a corpus. The mathematical formula is:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{IDF}(t, D) = \ln\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$$

Where $\text{TF}(t, d)$ is the frequency of term t in document d, N is the total documents in corpus D, and $\text{DF}(t)$ is the document frequency of term t.

Technical Intuition & Detailed Hand-Calculation

Consider a micro-corpus of $N = 2$ documents:

- d_1: `"cat mat"`
- d_2: `"mat rug"`
  Vocabulary: `["cat", "mat", "rug"]` (vocabulary size $|V| = 3$).

Step 1: Calculate Document Frequencies (DF) and Smooth IDF for each term:

- **"cat"**: $\text{DF}(\text{“cat”}) = 1$ (appears only in d_1)

$$\text{IDF}(\text{“cat”}) = \ln\left(\frac{1+2}{1+1}\right) + 1 = \ln(1.5) + 1 \approx 0.4055 + 1 = 1.4055$$

- **"mat"**: $\text{DF}(\text{“mat”}) = 2$ (appears in d_1 and d_2)

$$\text{IDF}(\text{“mat”}) = \ln\left(\frac{1+2}{1+2}\right) + 1 = \ln(1.0) + 1 = 0.0000 + 1 = 1.0000$$

- **"rug"**: $\text{DF}(\text{“rug”}) = 1$ (appears only in d_2)

$$\text{IDF}(\text{“rug”}) = \ln\left(\frac{1+2}{1+1}\right) + 1 = \ln(1.5) + 1 \approx 0.4055 + 1 = 1.4055$$

Step 2: Compute Raw TF-IDF Vector for d_1 (frequencies: $\text{TF}(\text{“cat”}) = 1$, $\text{TF}(\text{“mat”}) = 1$, $\text{TF}(\text{“rug”}) = 0$):

- $\text{TF-IDF}(\text{“cat”}, d_1) = 1 \times 1.4055 = 1.4055$
- $\text{TF-IDF}(\text{“mat”}, d_1) = 1 \times 1.0000 = 1.0000$
- $\text{TF-IDF}(\text{“rug”}, d_1) = 0 \times 1.4055 = 0.0000$

$$\mathbf{v}_{d_1} = [1.4055,\ 1.0000,\ 0.0000]^T$$

Step 3: Apply L_2 Normalization to prevent document length bias:

$$\|\mathbf{v}_{d_1}\|_2 = \sqrt{1.4055^2 + 1.0000^2 + 0.0^2} = \sqrt{1.9754 + 1.0000} = \sqrt{2.9754} \approx 1.7249$$

$$\mathbf{v}_{d_1,\text{norm}} = \left[\frac{1.4055}{1.7249}, \frac{1.0000}{1.7249}, 0.0\right]^T \approx [0.8148,\ 0.5797,\ 0.0]^T$$

Production Perspective & Trade-offs

- **Length Normalization**: Essential because longer documents naturally repeat words, inflating TF scores. L_2 normalization maps vectors onto a unit hypersphere, allowing Cosine Similarity to measure angles rather than absolute vector lengths.
- **Sparse Storage**: Large corpora yield $|V| > 100,000$, resulting in memory-heavy matrices. Production systems store representations in coordinate (COO) or compressed sparse row (CSR) formats to minimize storage overhead.

Follow-up Questions

- **Follow-up**: *How does a document frequency of 0 impact IDF?* -> Smooth IDF adds 1 to both the numerator and denominator, preventing division-by-zero errors.

Common Mistakes

- Forgetting to apply smooth factors or normalization, leading to document length bias.

Question 14: Compute cosine similarity between two TF-IDF vectors.

Short Interview Answer (30–60 seconds)

Cosine similarity measures the angle between two vectors:

$$\text{CosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}$$

If vectors are pre-normalized to unit length, this simplifies to the dot product.

Technical Intuition & Complexity

Given vectors:

- $\mathbf{a} = [0.8, 0.6, 0.0]$

- $\mathbf{b} = [0.0, 0.6, 0.8]$

$$\mathbf{a} \cdot \mathbf{b} = (0.8 \times 0) + (0.6 \times 0.6) + (0 \times 0.8) = 0.36$$

Production Perspective & Trade-offs

Dot products run efficiently on modern hardware using BLAS libraries, making cosine similarity comparisons fast in production.

Follow-up Questions

- **Follow-up**: *What is the cosine similarity of orthogonal vectors?* -> 0, indicating no shared token dimensions.

Common Mistakes

- Neglecting to normalize vectors, which biases similarity scores towards longer documents.

Question 15: Using the chain rule, derive the probability of a sentence in an N-gram language model.

Short Interview Answer (30–60 seconds)

By the chain rule, the joint probability of a sequence is:

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1})$$

An N-gram model simplifies this using the Markov assumption, limiting the context window to the preceding $N - 1$ words:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-N+1}, \ldots, w_{i-1})$$

Key Interview Points

- Joint probability factorization
- Markov context windows
- Conditional sequencing

Production Perspective & Trade-offs

Larger context orders N capture more dependencies but increase table memory footprints exponentially ($O(|V|^N)$).

Follow-up Questions

- **Follow-up**: *Why do we use log probabilities in evaluations?* -> Multiplying small values leads to numerical underflow; summing log probabilities keeps calculations stable.

Common Mistakes

- Forgetting the boundary markers (like `<s>` and `</s>`) when calculating sequence probabilities.

Question 16: Explain Maximum Likelihood Estimation (MLE) for N-gram language models.

Short Interview Answer (30–60 seconds)

MLE estimates sequence transitions by counting occurrences in a training corpus:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Where $C(w_{i-1}, w_i)$ is the bigram co-occurrence count.

Technical Intuition & Complexity

If the bigram `"sat on"` appears 10 times and `"sat"` appears 100 times, the MLE probability $P(\text{“on”} \mid \text{“sat”}) = 10/100 = 0.1$.

Production Perspective & Trade-offs

- **Time Complexity**: $O(1)$ constant time lookup if using precomputed n-gram tables.
- **Memory Complexity**: $O(|V|^N)$ space.

Follow-up Questions

- **Follow-up**: *What happens if a prefix is unseen during training?* -> The denominator becomes 0, crashing the calculation unless smoothing is applied.

Common Mistakes

- Assuming MLE generalizes well to unseen text without smoothing.

Question 17: Why is Laplace smoothing required? Compute smoothed probabilities for a simple example.

Short Interview Answer (30–60 seconds)

In Maximum Likelihood Estimation (MLE), any word transition unseen in the training set receives a probability score of 0. Due to the chain rule of probability, a single zero probability collapses the entire joint sequence probability to 0. Laplace smoothing (add-one smoothing) solves this by adding 1 to all counts, shifting probability mass from seen events to unseen events:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

Where $C(w_{i-1}, w_i)$ is the bigram count, $C(w_{i-1})$ is the unigram history count, and $|V|$ is the vocabulary size.

Technical Intuition & Detailed Hand-Calculation

Let's train a bigram language model on the micro-corpus:
`"the cat sat on the mat"` (contains 6 tokens).
Vocabulary: `["the", "cat", "sat", "on", "mat"]` (vocabulary size $|V| = 5$).

Step 1: Track counts for unigrams and bigrams:

- $C(\text{“the”}) = 2$
- $C(\text{“the”}, \text{“cat”}) = 1$
- $C(\text{“the”}, \text{“mat”}) = 1$
- $C(\text{“the”}, \text{“sat”}) = 0$ (unseen transition)

Step 2: Compute smoothed transition probabilities:

- **Seen transition** $P_{\text{Laplace}}(\text{“cat”} \mid \text{“the”})$:

$$P_{\text{Laplace}}(\text{“cat”} \mid \text{“the”}) = \frac{C(\text{“the”}, \text{“cat”}) + 1}{C(\text{“the”}) + |V|} = \frac{1+1}{2+5} = \frac{2}{7} \approx 0.2857$$

- **Unseen transition** $P_{\text{Laplace}}(\text{“sat”} \mid \text{“the”})$:

$$P_{\text{Laplace}}(\text{“sat”} \mid \text{“the”}) = \frac{C(\text{“the”}, \text{“sat”}) + 1}{C(\text{“the”}) + |V|} = \frac{0+1}{2+5} = \frac{1}{7} \approx 0.1429$$

Notice that the sum of probabilities across all possible next tokens for the history `"the"` remains normalized:

$$\sum_{w \in V} P(w \mid \text{“the”}) = \frac{2}{7}(\text{for “cat”}) + \frac{2}{7}(\text{for “mat”}) + \frac{1}{7}(\text{for “sat”}) + \frac{1}{7}(\text{for “on”}) + \frac{1}{7}(\text{for ‘}$$

Production Perspective & Trade-offs

- **The $|V|$ Bottleneck**: While Laplace smoothing ensures mathematical validity, adding 1 to all unseen transitions allocates a massive amount of probability mass to the long tail of rare or unseen words in large vocabularies (e.g. $|V| = 50,000$), significantly degrading the probability of common, natural transitions.
- **Production Solution**: Modern NLP systems rely on Absolute Discounting or Kneser-Ney smoothing, which discount a fixed value d from seen counts and distribute it proportionally based on how likely a word is to complete unseen context (continuation probability).

Follow-up Questions

- **Follow-up**: *How does Kneser-Ney smoothing improve on Laplace?* -> Kneser-Ney uses absolute discounting and a continuation probability to estimate how likely a word is to complete an unseen context.

Common Mistakes

- Forgetting to add vocabulary size $|V|$ to the denominator, which violates probability normalization rules.

Question 18: What is Perplexity? Derive its mathematical formulation and explain its intuition.

Short Interview Answer (30–60 seconds)

Perplexity (PPL) measures how well a probability model predicts a sample. Intuitively, it represents the average branching factor—the number of equally likely words the model is choosing from at each step. Mathematically, it is the exponentiated cross-entropy loss of the sequence:

$$\text{PPL}(W) = P(w_1, w_2, \ldots, w_m)^{-\frac{1}{m}} = e^{\mathcal{L}_{\text{CE}}}$$

Where m is the sequence length, $P(w_1, \ldots, w_m)$ is the sequence joint probability, and $\mathcal{L}_{\text{CE}}$ is the average cross-entropy loss per token.

Technical Intuition & Detailed Hand-Calculation

Let's derive perplexity from cross-entropy and compute it for a tiny 3-token sequence:

1. Mathematical Derivation:

The cross-entropy loss per token for a sequence W of length m is:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m}\sum_{i=1}^{m}\ln P(w_i \mid w_{1:i-1}) = -\frac{1}{m}\ln P(w_1,\ldots,w_m)$$

Exponentiating both sides:

$$e^{\mathcal{L}_{\text{CE}}} = e^{-\frac{1}{m}\ln P(W)} = \left(e^{\ln P(W)}\right)^{-\frac{1}{m}} = P(W)^{-\frac{1}{m}} = \text{PPL}(W)$$

2. Step-by-Step Hand-Calculation:

Suppose our bigram model processes a test sequence $W = (w_1, w_2, w_3)$ with the following conditional probabilities:

- $P(w_1) = 0.50$
- $P(w_2 \mid w_1) = 0.20$
- $P(w_3 \mid w_2) = 0.40$
- **Step 1: Compute Joint Probability:**

$$P(W) = P(w_1) \times P(w_2 \mid w_1) \times P(w_3 \mid w_2) = 0.50 \times 0.20 \times 0.40 = 0.0400$$

- **Step 2: Calculate Average Cross-Entropy Loss ($\mathcal{L}_{\text{CE}}$):**

$$\mathcal{L}_{\text{CE}} = -\frac{1}{3}\ln(0.0400) \approx -\frac{1}{3} \times (-3.2189) = 1.0730$$

- **Step 3: Calculate Perplexity:**

$$\text{PPL}(W) = e^{1.0730} = 0.0400^{-\frac{1}{3}} = 25^{\frac{1}{3}} \approx 2.9240$$

Findings & Interpretation:

The perplexity of 2.9240 means that at each token prediction step, the model was as confused as if it had to choose uniformly from ≈ 2.92 candidate words. A lower perplexity indicates a more confident model.

Production Perspective & Trade-offs

- **Comparison Limits**: Perplexity can only be used to compare models that share the exact same tokenizer and vocabulary size. A model with a vocabulary size $|V| = 1,000$ will inherently have a lower baseline perplexity than a model with $|V| = 50,000$, even if the latter is semantically superior, because it is choosing from fewer alternatives.

- **Out of Vocabulary (OOV)**: If a tokenizer fails to handle OOV tokens, a single unseen token can make $P(w_i) = 0$, causing PPL to explode to infinity, requiring clipping or smoothing.

Follow-up Questions

- **Follow-up**: *How does vocabulary size impact perplexity comparisons?* -> Models with smaller vocabularies inherently yield lower perplexity scores because they choose from fewer options.

Common Mistakes

- Comparing perplexity scores across models that use different tokenizers or vocabularies.

Question 19: Explain how CBOW and Skip-gram learn word embeddings.

Short Interview Answer (30–60 seconds)

Word2Vec trains static embeddings using local context predictions:

- **CBOW**: Predicts a target word given its surrounding context words.
- **Skip-gram**: Predicts context words given a target word.

Technical Intuition & Complexity

- **CBOW**: Context vectors are averaged into a single vector $\mathbf{h}$ to predict the target.
- **Skip-gram**: Runs multiple predictions per target word, making it slower to train but yielding better representations for rare tokens.

Production Perspective & Trade-offs

- **CBOW**: Fast to train, but averages out context details.
- **Skip-gram**: Slower to train, but captures context details and rare tokens well.

Follow-up Questions

- **Follow-up**: *What optimization algorithms speed up Word2Vec?* -> Negative Sampling and Hierarchical Softmax.

Common Mistakes

- Assuming Word2Vec requires labeled training data (it is self-supervised).

Question 20: Why does Negative Sampling make Word2Vec training faster?

Short Interview Answer (30–60 seconds)

Standard multiclass Softmax requires computing a normalization sum over the entire vocabulary $|V|$ (often $> 100,000$ tokens) in the denominator for every single training step, which is computationally prohibitive. Skip-gram with Negative Sampling (SGNS) reformulates the task as a binary logistic regression game. For each target-context pair, the model optimizes a binary classifier to distinguish the true target-context pair from K randomly drawn "noise" (negative) samples, reducing computing complexity from $O(|V|)$ to $O(K)$.

The SGNS loss function is:

$$\mathcal{L}_{\text{SGNS}} = -\ln \sigma(\mathbf{v}_w \cdot \mathbf{v}'_{w_c}) - \sum_{i=1}^{K} \ln \sigma(-\mathbf{v}_w \cdot \mathbf{v}'_{w_i})$$

Technical Intuition & Detailed Hand-Calculation

Let's compute the negative sampling loss for a single step with a key target word and $K = 1$ negative sample:

- Target word: `"cat"` (context embedding vector $\mathbf{v}_{\text{cat}} = [0.10,\ 0.80,\ -0.20]^T$)
- Positive context word: `"sat"` (target embedding vector $\mathbf{v}'_{\text{sat}} = [0.20,\ 0.90,\ 0.10]^T$)
- Negative noise word: `"refrigerator"` (target embedding vector $\mathbf{v}'_{\text{refrig}} = [-0.90,\ 0.10,\ 0.80]^T$)

Step 1: Compute Dot Products:

- Positive pair dot product:

  $\mathbf{v}_{\text{cat}} \cdot \mathbf{v}'_{\text{sat}} = (0.10 \times 0.20) + (0.80 \times 0.90) + (-0.20 \times 0.10) = 0.0200 + 0.7200 - 0.0$

- Negative pair dot product:

  $\mathbf{v}_{\text{cat}} \cdot \mathbf{v}'_{\text{refrig}} = (0.10 \times -0.90) + (0.80 \times 0.10) + (-0.20 \times 0.80) = -0.0900 + 0.0800$

Step 2: Compute Logistic Sigmoid Probabilities ($\sigma(x) = \frac{1}{1+e^{-x}}$):

- Positive pair probability:

$$\sigma(0.7200) = \frac{1}{1 + e^{-0.7200}} \approx \frac{1}{1 + 0.4868} = 0.6726$$

- Negative pair probability (note the negative sign $-\mathbf{v} \cdot \mathbf{v}'$):

$$\sigma(-(-0.1700)) = \sigma(0.1700) = \frac{1}{1 + e^{-0.1700}} \approx \frac{1}{1 + 0.8437} = 0.5424$$

Step 3: Calculate the SGNS Loss:

$$\mathcal{L}_{\text{SGNS}} = -\ln(0.6726) - \ln(0.5424) \approx 0.3966 + 0.6117 = 1.0083$$

Findings & Interpretation:

The loss score is 1.0083. During backpropagation, the gradients will adjust only the vectors for `"cat"`, `"sat"`, and `"refrigerator"`. The other $|V| - 3$ word vectors are completely untouched during this step, which is why it runs orders of magnitude faster.

Production Perspective & Trade-offs

- **Time Complexity**: Reduces weight-update calculations from $O(|V|)$ to $O(K)$.
- **Negative Sampling Distribution**: Negatives are sampled from a smoothed unigram distribution:

$$P_n(w) \propto U(w)^{0.75}$$

  Raising unigram frequency $U(w)$ to the power of 0.75 boosts the probability of sampling rare words (e.g. if $U(w_1) = 0.99$ and $U(w_2) = 0.01$, the ratio shifts from $99:1$ to $\approx 32:1$), preventing the model from over-optimizing on common stop words (like `"the"` or `"is"`).

Follow-up Questions

- **Follow-up**: *What is the optimal value for K?* -> Typically 5–20 for small datasets, and 2–5 for large corpora.

Common Mistakes

- Believing negative sampling updates all vocabulary weights at each step.

Question 21: Explain mathematically why RNNs suffer from vanishing gradients.

Short Interview Answer (30–60 seconds)

During backpropagation through time (BPTT), gradients are scaled by the recurrent weight matrix product:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \text{diag}(1 - h_k^2) W_{hh}^T$$

If the largest eigenvalue of W_{hh} is less than 1, multiplying this matrix repeatedly over a long sequence causes the gradient to shrink exponentially to 0.

Key Interview Points

- Backpropagation through time (BPTT)
- Jacobian chain product
- Spectral radius $\rho(W_{hh}) < 1$

Production Perspective & Trade-offs

Vanishing gradients prevent standard RNNs from learning long-range dependencies, requiring the use of LSTMs or GRUs.

Follow-up Questions

- **Follow-up**: *How does gradient clipping resolve exploding gradients?* -> If the gradient norm exceeds a threshold, it is scaled down: $\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{\|\mathbf{g}\|}$.

Common Mistakes

- Confusing vanishing gradients with dead ReLU units.

Question 22: Explain how LSTM's cell state mitigates the vanishing gradient problem.

Short Interview Answer (30–60 seconds)

Standard RNNs propagate gradients by multiplying the error vector repeatedly by the recurrent hidden-to-hidden weight matrix $\mathbf{W}_{hh}$ at each backpropagation step. This multiplicative chain leads to vanishing gradients if the eigenvalues of $\mathbf{W}_{hh}$ are less than 1. In contrast, LSTMs propagate error gradients through an additive **Constant Error Carousel (CEC)** located in the cell state $\mathbf{C}_t$. The cell state update equation is linear:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Since the update is additive, the derivative $\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}}$ has a direct linear pathway (scaled by forget gate $\mathbf{f}_t$), bypassing recurrent weight matrix multiplication.

Technical Intuition & Detailed Derivation

To see how LSTM mitigates vanishing gradients, let's derive the gradient pathway back from cell state $\mathbf{C}_t$ to $\mathbf{C}_{t-1}$:

1. Derive the Cell State Jacobian:

The cell state update is:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Differentiating $\mathbf{C}_t$ with respect to $\mathbf{C}_{t-1}$ yields:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \mathbf{f}_t + \mathbf{C}_{t-1} \odot \frac{\partial \mathbf{f}_t}{\partial \mathbf{C}_{t-1}} + \tilde{\mathbf{C}}_t \odot \frac{\partial \mathbf{i}_t}{\partial \mathbf{C}_{t-1}} + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{C}}_t}{\partial \mathbf{C}_{t-1}}$$

2. The CEC Shortcut:

If we assume that the forget gate is fully open ($\mathbf{f}_t \approx \mathbf{1}$) and the hidden states are structured such that the gate derivatives (the latter three terms in the sum) are close to 0:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t \approx \mathbf{1}$$

3. Propagating over $T - t$ steps:

$$\frac{\partial \mathbf{C}_T}{\partial \mathbf{C}_t} = \prod_{k=t+1}^{T} \frac{\partial \mathbf{C}_k}{\partial \mathbf{C}_{k-1}} \approx \prod_{k=t+1}^{T} \mathbf{f}_k$$

If $\mathbf{f}_k \approx 1$ (the forget gate remains open), the error gradient propagates back across arbitrarily long sequence lengths without decaying. This additive, gated shortcut is called the **Constant Error Carousel (CEC)**.

Production Perspective & Trade-offs

- **Forget Gate Calibration**: The forget gate $\mathbf{f}_t$ is crucial. If the model is not trained well and $\mathbf{f}_t \to 0$, gradients will vanish just as in standard RNNs.
- **Compute Overhead**: The gating mechanism requires four separate linear layers per cell ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t, \tilde{\mathbf{C}}_t$), quadrupling the parameter count and computational budget compared to standard RNNs. In high-throughput production settings, engineers often use GRUs (Gate Recurrent Units) which merge cell and hidden states to save 25% on parameter size.

Follow-up Questions

- **Follow-up**: *What happens if the forget gate is always 0?* -> The model discards all historical cell state information at each step, behaving like a standard feedforward network.

Common Mistakes

- Believing LSTMs completely eliminate vanishing gradients (they can still vanish if the forget gate outputs 0).

---

3. Production & Engineering Questions (23-30)

Question 23: What challenges arise when deploying NLP models to production?

Short Interview Answer (30–60 seconds)

Production challenges include: managing GPU/CPU latency budgets (especially for autoregressive text generation), handling vocabulary drift, maintaining preprocessing consistency between training and serving, and managing deployment costs.

Key Interview Points

- Latency budgets
- Vocabulary drift
- Model quantization

Production Perspective & Trade-offs

Deploying models requires balancing latency and accuracy. Quantization reduces VRAM footprints but can degrade accuracy on edge cases.

Follow-up Questions

- **Follow-up**: *How does model pruning affect execution speed?* -> Pruning zeroes out weights to create sparse matrices, which can bypass calculations on hardware that supports sparse operations.

Common Mistakes

- Assuming research accuracies translate directly to production without latency optimization.

Question 24: How do you handle vocabulary drift in production systems?

Short Interview Answer (30–60 seconds)

Vocabulary drift occurs when user inputs introduce new words or symbols (e.g. slang, emojis) not present in the model's vocabulary. We handle this by using byte-fallback tokenizers, retraining models on live data, and maintaining dynamic lookup dictionary expansions.

Key Interview Points

- Byte-fallback tokenization
- Retraining loops
- Dynamic dictionary expansions

Production Perspective & Trade-offs

Byte-fallback tokenizers decompose unknown words down to raw bytes, preventing `<unk>` errors at the cost of slightly longer sequence lengths.

Follow-up Questions

- **Follow-up**: *How does vocabulary size impact model serving cost?* -> A larger vocabulary increases the classification layer's size, increasing GPU VRAM usage.

Common Mistakes

- Relying on manual dictionary additions, which fails to scale in dynamic environments.

Question 25: What is training-serving skew? Why is it problematic?

Short Interview Answer (30–60 seconds)

Training-serving skew occurs when the preprocessing pipeline, data distribution, or environment features differ between the training phase and the production serving layer, leading to model degradation.

Key Interview Points

- Preprocessing consistency
- Feature drift
- Pipeline alignment

Production Perspective & Trade-offs

To prevent skew, wrap tokenizers and preprocessing code into a shared, version-controlled library used by both training and serving pipelines.

Follow-up Questions

- **Follow-up**: *How does Unicode decomposition cause training-serving skew?* -> If the serving tokenizer composition doesn't match the training setup, words split differently, mapping to incorrect token indices.

Common Mistakes

- Using separate codebases (e.g., Python for training, C++ for serving) without strict testing against index outputs.

Question 26: Explain Data Drift versus Concept Drift with examples.

Short Interview Answer (30–60 seconds)

- **Data Drift**: The input data distribution changes, but input-to-label relationships remain the same.
- **Concept Drift**: The mapping relationship between inputs and labels shifts over time.

Key Interview Points

- Covariate shift
- Semantic label drift
- Population monitoring

Production Perspective & Trade-offs

- **Data Drift Example**: A sentiment classifier trained on news articles processes social media comments containing slang.
- **Concept Drift Example**: The term `"viral"` shifts from representing a negative healthcare quarantine tag (2019) to a positive marketing indicator (2021).

Follow-up Questions

- **Follow-up**: *How do you monitor for concept drift?* -> By tracking performance metrics (like F1 score or accuracy) on labeled production data over time.

Common Mistakes

- Retraining models on raw inputs to resolve concept drift without verifying updated label mappings.

Question 27: When would you choose batch inference over real-time inference?

Short Interview Answer (30–60 seconds)

Choose **Batch Inference** when throughput is the main priority and real-time response latency is not required (e.g. running classification sweeps over nightly logs). Choose **Real-Time Inference** when processing interactive streaming user inputs with strict latency limits (e.g. search query auto-completion).

Key Interview Points

- High throughput vs. Low latency
- GPU utilization profiles
- Cost optimization

Production Perspective & Trade-offs

Batch inference maximizes GPU utilization by processing large chunks of data in parallel, lowering cost per token. Real-time inference processes single inputs, which can leave GPUs underutilized.

Follow-up Questions

- **Follow-up**: *How does dynamic batching optimize real-time inference?* -> It groups incoming concurrent user requests into a single batch at the server level, increasing throughput.

Common Mistakes

- Deploying real-time servers for tasks that could be run as cheaper batch pipelines.

Question 28: How do quantization and pruning reduce inference cost?

Short Interview Answer (30–60 seconds)

- **Quantization**: Converts model weights from float32 (32-bit) to lower-precision formats like float16 or int8, reducing VRAM footprint and memory bandwidth limits.

- **Pruning**: Zeroes out non-essential parameters, creating sparse matrices that can speed up inference.

Key Interview Points

- Precision conversion (int8)
- Weight sparsity
- VRAM footprint compression

Production Perspective & Trade-offs

Quantization reduces VRAM usage by up to 75%, allowing models to run on cheaper hardware, but can degrade accuracy on edge cases.

Follow-up Questions

- **Follow-up**: *What is Post-Training Quantization (PTQ) vs. Quantization-Aware Training (QAT)? ->* PTQ quantizes weights after training, while QAT models precision loss during training, yielding better accuracy.

Common Mistakes

- Assuming pruning always speeds up models (it requires hardware that supports sparse operations to yield speed gains).

Question 29: What preprocessing inconsistencies can lead to degraded model performance?

Short Interview Answer (30–60 seconds)

Inconsistencies include: using different lowercase rules, different regex string cleanups, different Unicode normalizations (NFC vs. NFD), or different subword vocabularies between training and serving.

Key Interview Points

- Token normalization mismatch
- Unicode normalizations
- Pipeline testing

Production Perspective & Trade-offs

A single preprocessing mismatch (like missing punctuation cleaning) can cause the tokenizer to split words differently, mapping them to incorrect vectors.

Follow-up Questions

- **Follow-up**: *How does regex cleaning impact latency?* -> Complex regex lookaheads run on CPU and can become a bottleneck in high-throughput pipelines.

Common Mistakes

- Assuming standard tokenizers handle raw, uncleaned text consistently without normalizations.

Question 30: What monitoring metrics would you track for an NLP model in production?

Short Interview Answer (30–60 seconds)

Track: **Systems Metrics** (inference latency, GPU VRAM usage, error rates), **Data Metrics** (out-of-vocabulary token rates, input length distributions, data drift metrics), and **Performance Metrics** (prediction confidence distributions, feedback classifications).

Key Interview Points

- GPU/VRAM health
- Data drift distributions
- OOV token rates

Production Perspective & Trade-offs

Set up automated alarms on OOV rates. A spike in OOV tokens indicates data drift, signaling that the model needs retraining.

Follow-up Questions

- **Follow-up**: *How do you measure drift on text embeddings?* -> By tracking cosine distance distributions between production embeddings and training reference vectors over time.

Common Mistakes

- Monitoring only systems metrics (like CPU load) while neglecting data drift and model accuracy decay.

---

4. Debugging & Evaluation Questions (31-35)

Question 31: Why can BLEU and ROUGE give misleading evaluation results?

Short Interview Answer (30–60 seconds)

BLEU and ROUGE measure exact n-gram overlaps. They suffer from semantic blind spots: they score synonyms as zero matches (e.g. `"feline"` vs `"cat"`) and can assign high scores to grammatically similar but logically opposite generations (such as negations).

Key Interview Points

- N-gram overlap limits
- Synonym blind spots
- Negation mismatch

Production Perspective & Trade-offs

Using only BLEU/ROUGE can lead to deploying models that output grammatically correct but factually incorrect summaries. Use semantic metrics like BERTScore alongside human validation loops.

Follow-up Questions

- **Follow-up**: *How does METEOR improve on BLEU?* -> METEOR incorporates stem matching and synonyms using dictionary databases (like WordNet) to compute alignments.

Common Mistakes

- Believing a high BLEU score guarantees factual correctness in text generation.

Question 32: When would you prefer BERTScore over BLEU?

Short Interview Answer (30–60 seconds)

Prefer **BERTScore** when evaluating semantic similarity, paraphrasing, or tasks where synonyms (e.g. `"feline"` instead of `"cat"`) are acceptable, because exact-match n-gram metrics (BLEU, ROUGE) assign a score of 0 to synonyms. Prefer **BLEU** or **ROUGE** for machine translation regression testing where exact vocabulary matches or speed are critical. BERTScore computes greedy cosine alignments over contextual embeddings from a pre-trained language model, capturing semantic shifts that overlap metrics miss.

Technical Intuition & Detailed Hand-Calculation

Let's compute BERTScore greedy alignment scores for:

- Candidate (c): `"A feline rested on the rug"` (Length $c = 6$)
- Reference (r): `"The cat sat on the mat"` (Length $r = 6$)

Suppose our contextual embedding model generates vectors for each token, and the maximum cosine similarity scores between candidate and reference tokens are:

- `"A"` matches best with `"The"` (Similarity = 0.12)
- `"feline"` matches best with `"cat"` (Similarity = 0.88)
- `"rested"` matches best with `"sat"` (Similarity = 0.82)
- `"on"` matches best with `"on"` (Similarity = 0.95)
- `"the"` matches best with `"the"` (Similarity = 0.98)
- `"rug"` matches best with `"mat"` (Similarity = 0.85)

1. Calculate BERTScore Recall (R_{BERT}):

Aligns each reference token to the best matching candidate token, normalized by reference length:

$$R_{\text{BERT}} = \frac{1}{|r|} \sum_{w_j \in r} \max_{w_i \in c} \mathbf{E}(w_i)^T \mathbf{E}(w_j) = \frac{0.12 + 0.88 + 0.82 + 0.95 + 0.98 + 0.85}{6} = \frac{4.60}{6} \approx$$

2. Calculate BERTScore Precision (P_{BERT}):

Aligns each candidate token to the best matching reference token, normalized by candidate length:

$$P_{\text{BERT}} = \frac{1}{|c|} \sum_{w_i \in c} \max_{w_j \in r} \mathbf{E}(w_i)^T \mathbf{E}(w_j) = \frac{0.12 + 0.88 + 0.82 + 0.95 + 0.98 + 0.85}{6} = \frac{4.60}{6} \approx$$

3. Calculate BERTScore F1 (F_{BERT}):

$$F_{\text{BERT}} = 2 \times \frac{P_{\text{BERT}} \times R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \approx 2 \times \frac{0.7667 \times 0.7667}{0.7667 + 0.7667} \approx 0.7667$$

Findings & Interpretation:

While BLEU-2 would penalize this candidate heavily due to lack of exact word matches (BLEU unigram overlap would fail on `"feline"` vs `"cat"` and `"rug"` vs `"mat"`), BERTScore detects the semantic equivalence of `"feline" \leftrightarrow "cat"` (0.88) and `"rug" \leftrightarrow "mat"` (0.85), yielding a high F1 score of 0.7667, capturing synonym alignments.

Production Perspective & Trade-offs

- **Inference Latency**: BLEU is a string overlap check running in $O(L_c \cdot L_r)$ on CPU (taking $< 0.1\text{ms}$). BERTScore requires running a transformer encoder forward-pass on GPU to extract token embeddings, increasing computing cost and latency by orders of magnitude, making it expensive for real-time model evaluation APIs.
- **Model Bias**: BERTScore scores depend on the underlying embedding model. A model biased against certain structures can skew evaluation scores, requiring careful selection of the encoder (e.g. RoBERTa-large).

Follow-up Questions

- **Follow-up**: *How does BERTScore calculate greedy alignments?* -> It computes cosine similarities between all candidate and reference token embeddings, matching each word to its highest scoring semantic counterpart.

Common Mistakes

- Neglecting the compute cost of running BERTScore over large evaluation datasets.

Question 33: How would you debug an NLP classifier with poor precision but high recall?

Short Interview Answer (30–60 seconds)

Poor precision but high recall means the classifier is over-predicting the positive class, resulting in many false positives. To debug, adjust the prediction threshold (increase the probability cutoff for the positive class), analyze the confusion matrix, and address class imbalances in the training data.

Key Interview Points

- High False Positives
- Positive threshold adjustments
- Imbalanced training data

Production Perspective & Trade-offs

In spam filtering, high recall with low precision means the model catches all spam but filters out valid emails. Adjust prediction thresholds to prioritize precision.

Follow-up Questions

- **Follow-up**: *How does F1 score help evaluate this trade-off?* -> The F1 score is the harmonic mean of precision and recall, helping you find a balance between the two metrics.

Common Mistakes

- Retraining the model from scratch without first adjusting prediction thresholds.

Question 34: How would you diagnose exploding gradients during RNN training?

Short Interview Answer (30–60 seconds)

Exploding gradients show up as training loss spiking to `NaN`, weight parameters diverging, or gradients showing extremely large norms during backpropagation. Diagnose this by monitoring gradient norms at each step. Fix it by applying gradient clipping or reducing the learning rate.

Key Interview Points

- Loss spikes to NaN
- Gradient norm monitors
- Gradient clipping

Production Perspective & Trade-offs

Gradient clipping prevents exploding gradients but does not address vanishing gradients. Use LSTMs or GRUs to handle vanishing gradients.

Follow-up Questions

- **Follow-up**: *What max norm value is typically used for clipping?* -> A max norm threshold of 1.0 is standard in deep learning pipelines.

Common Mistakes

- Assuming `NaN` loss is always caused by division-by-zero errors in the data rather than exploding gradients.

Question 35: How would you perform error analysis for an NLP system?

Short Interview Answer (30–60 seconds)

Perform error analysis by: logging classification outputs, filtering for predictions where ground-truth and prediction labels mismatch, manually inspecting a representative sample (e.g. 100-200 errors), categorizing errors into groups (e.g., tokenization errors, typos, class bias), and implementing targeted training data updates or preprocessing rules to resolve them.

Key Interview Points

- Mismatch logs audits
- Manual error labeling
- System retrain patches

Production Perspective & Trade-offs

Error analysis helps you identify specific preprocessing bugs or data drift patterns, allowing you to implement targeted fixes instead of blindly retraining the model.

Follow-up Questions

- **Follow-up**: *How does class imbalance impact error analysis?* -> It often causes the model to over-predict the majority class, resulting in high rates of false negatives for the minority class.

Common Mistakes

- Relying only on aggregate metrics (like accuracy) without auditing individual error logs.

---

5. System Design & Applied Questions (36-40)

Question 36: Design a spam email classifier for millions of users.

Short Interview Answer (30–60 seconds)

The system uses: a preprocessing pipeline to normalize HTML and emails $\rightarrow$ Feature Hashing (to maintain a zero-memory vocabulary footprint) $\rightarrow$ a fast classifier model (like logistic regression or Naïve Bayes) for initial filtering $\rightarrow$ a secondary model (like an LSTM or transformer) for low-confidence queries.

Key Interview Points

- High throughput pipeline

- Feature Hashing trick
- Multi-tier classification

Production Perspective & Trade-offs

- **High throughput**: Use Feature Hashing to handle high volume without storing massive lookup dictionaries.
- **Low latency**: Filter obvious spam early using cheap models, routing only ambiguous emails to resource-intensive classification models.
- **Drift**: Monitor OOV rates and user spam flags to detect vocabulary drift.

Follow-up Questions

- **Follow-up**: *How do you handle attachments?* -> Extract text from attachments using parser libraries, or route them through separate file scanner pipelines.

Common Mistakes

- Proposing heavy, slow transformer models for the entire email volume, which incurs high compute cost and latency.

Question 37: Design an autocomplete/search suggestion system.

Short Interview Answer (30–60 seconds)

The system uses: a trie structure populated with query probabilities. For the current prefix, the system looks up the top K completions with the highest MLE probabilities. Smooth prediction probabilities using Katz backoff or interpolation to handle rare or unseen prefixes.

Key Interview Points

- Trie prefix lookups
- MLE probabilities
- Katz backoff smoothing

Production Perspective & Trade-offs

- **Low Latency**: Suggestion lookups must run in <10ms. Store the prefix trie in memory (e.g. using Redis), caching frequent suggestions.
- **Storage**: Prune n-grams with count frequencies < 5 to compress trie memory size.

Follow-up Questions

- **Follow-up**: *How do you personalize suggestions?* -> By weighting the trie path probabilities based on the user's historical search categories.

Common Mistakes

- Querying SQL databases directly for auto-complete lookups, which is too slow for real-time systems.

Question 38: Design a sentiment analysis pipeline for social media.

Short Interview Answer (30–60 seconds)

The system uses: a preprocessing step that retains emojis and punctuation (cues for sentiment) $\rightarrow$ subword tokenization (BPE) $\rightarrow$ a bidirectional classifier model $\rightarrow$ metric logging. Monitor for data drift using population stability checks.

Key Interview Points

- Emoji/slang preservation
- Bidirectional context
- Data drift monitoring

Production Perspective & Trade-offs

Social media text features high rates of typos, slang, and emojis. Retaining emojis and using subword tokenization is critical to capture sentiment cues. Monitor data drift closely, as vocabulary shifts quickly on social platforms.

Follow-up Questions

- **Follow-up**: *Why use a bidirectional model?* -> It captures negation words (like `"not"`) that appear before or after the target word, resolving local context.

Common Mistakes

- Discarding emojis and punctuation during preprocessing, which removes critical sentiment signals.

Question 39: Design an FAQ chatbot using classical NLP techniques (without LLMs).

Short Interview Answer (30–60 seconds)

The system matches queries against a database of FAQs: pre-process user query → compute TF-IDF representation vector → query the FAQ database using BM25 → select the answer associated with the highest similarity score.

Key Interview Points

- BM25 keyword matching
- Similarity ranking
- Preprocessing normalizations

Production Perspective & Trade-offs

- **Advantages**: Fast, deterministic, runs on CPU with minimal hosting costs.
- **Disadvantages**: Cannot handle paraphrasing or queries that use synonyms.
- **Fix**: Use Sentence-BERT to generate semantic embeddings for queries, combining BM25 keyword matching and semantic search.

Follow-up Questions

- **Follow-up**: *How do you handle spelling errors?* -> Apply spelling correction algorithms or use subword n-grams (FastText) to compute similarities.

Common Mistakes

- Proposing heavy generative models when simple query-matching retrieval systems are sufficient.

Question 40: Given an NLP application, how would you decide between: TF-IDF, Word2Vec, FastText, LSTM, Transformer?

Short Interview Answer (30–60 seconds)

Select based on accuracy requirements, latency budgets, and sequence complexity:

- **TF-IDF**: Best for simple keyword searches or classification on static text with tight compute budgets.
- **Word2Vec**: Best for semantic similarity lookups on static vocabularies.

- **FastText**: Best when handling out-of-vocabulary (OOV) tokens, typos, or morphologically rich languages.
- **LSTM**: Best for processing sequence structures when context length is moderate and training resources are limited.
- **Transformer**: Best when accuracy is the main priority and you have the compute resources to support parallel training and long contexts.

Key Interview Points

- Latency vs. Accuracy
- OOV handling needs
- Compute resources

Production Perspective & Trade-offs

In production, start with a cheap model (TF-IDF or Word2Vec) to establish a baseline. Upgrade to LSTMs or Transformers only if the accuracy gains justify the higher latency and hosting costs.

Follow-up Questions

- **Follow-up**: *Which model handles multilingual text best?* -> FastText or Transformers, as they decompose text into subwords, reducing the size of multilingual vocabulary tables.

Common Mistakes

- Defaulting to complex Transformer models without evaluating simpler baselines first.

---

6. Advanced Questions (41-50)

Question 41: Why is BM25 generally better than TF-IDF for retrieval?

Short Interview Answer (30–60 seconds)

BM25 improves on TF-IDF by scaling term frequency non-linearly to prevent saturation and penalizing long documents that contain terms simply due to size:

- **TF-IDF**: Scales term frequency linearly, allowing a term repeated 100 times to dominate the score.
- **BM25**: Uses a saturation parameter k_1 to bound the score contribution of any single term, and normalizes by document length using parameter b.

Key Interview Points

- Term frequency saturation (k_1)
- Document length normalization (b)
- Sub-linear scaling

Production Perspective & Trade-offs

BM25 is standard in search engines (like Elasticsearch) because it prevents long documents containing query terms from dominating search rankings.

Follow-up Questions

- **Follow-up**: *What is the effect of setting parameter b to 0?* -> Length normalization is deactivated; document length has no impact on similarity scoring.

Common Mistakes

- Believing BM25 requires deep neural network evaluations (it is a keyword count algorithm).

Question 42: Why does FastText outperform Word2Vec for rare words?

Short Interview Answer (30–60 seconds)

Word2Vec learns independent vectors for each word, meaning rare words have poorly trained embeddings due to insufficient context samples. FastText represents words as a bag of character n-grams, allowing rare words to inherit vector representations from shared subwords, improving semantic alignment.

Key Interview Points

- Character n-gram embeddings
- Shared root semantics
- Typo resilience

Production Perspective & Trade-offs

FastText is resilient against typos and out-of-vocabulary (OOV) terms, making it useful in production environments with noisy user inputs.

Follow-up Questions

- **Follow-up**: *Does FastText require more memory than Word2Vec?* -> Yes, because it must store embeddings for both words and character n-grams.

Common Mistakes

- Assuming FastText is as slow as context-dependent transformer models (it is still a static lookup table model).

Question 43: Why are bidirectional RNNs unsuitable for autoregressive text generation?

Short Interview Answer (30–60 seconds)

Autoregressive generation generates text sequentially (token by token). Bidirectional RNNs require the entire input sequence to compute backward hidden states. Consequently, they cannot generate text because future tokens are not yet known.

Key Interview Points

- Autoregressive generation
- Future context dependency
- Causal masking

Production Perspective & Trade-offs

For generation tasks, use causal decoder models (which use causal masking to prevent the model from looking ahead) to ensure step-by-step token generation.

Follow-up Questions

- **Follow-up**: *Where are bidirectional models useful?* -> In sequence tagging (NER, POS tagging) and text representation (BERT), where the entire input sentence is available.

Common Mistakes

- Proposing bidirectional models (like BERT) for text generation tasks.

Question 44: How does teacher forcing affect Seq2Seq training?

Short Interview Answer (30–60 seconds)

During training, Teacher Forcing feeds the ground-truth target tokens as input to the decoder instead of the model's own previous predictions. This prevents early prediction errors from propagating through the sequence, stabilizing and speeding up early training.

Key Interview Points

- Ground-truth target inputs
- Training sequence alignment
- Exposure bias

Production Perspective & Trade-offs

- **Advantages**: Speeds up convergence during training.
- **Disadvantages**: Introduces **Exposure Bias** because the model never encounters its own errors during training, leading to generation errors during production inference.

Follow-up Questions

- **Follow-up**: *How do you mitigate exposure bias?* -> Using scheduled sampling, where you gradually transition from ground-truth inputs to the model's own predictions as training progresses.

Common Mistakes

- Using teacher forcing during inference (when ground-truth targets are not available).

Question 45: Explain greedy decoding versus beam search.

Short Interview Answer (30–60 seconds)

- **Greedy Search**: Selects the single most likely token at each step ($y_t = \arg\max P(y \mid y_{<t})$). It is computationally fast ($O(L)$) but cannot backtrack.
- **Beam Search**: Explores multiple paths ($O(L \cdot B)$), keeping a running set of the B most likely sequences. It improves generation quality at the cost of higher latency and memory usage.

Key Interview Points

- Local vs. Global optimization
- Beam width parameter (B)
- Latency trade-offs

Production Perspective & Trade-offs

In production, large beam sizes (e.g. $B > 10$) increase latency and memory usage. A beam size of 2–5 is typically preferred to balance speed and quality.

Follow-up Questions

- **Follow-up**: *Why apply length normalization in beam search?* -> Without normalization, beam search favors shorter generations because multiplying probabilities yields smaller values as sequence length increases.

Common Mistakes

- Believing beam search guaranteed to find the globally optimal sequence (it is still a heuristic search).

Question 46: What is exposure bias in Seq2Seq models?

Short Interview Answer (30–60 seconds)

Exposure bias occurs when a model is trained using teacher forcing (receiving ground-truth inputs) but is evaluated during inference by feeding its own predictions back as input. Early prediction errors propagate, causing the model to generate low-quality text.

Key Interview Points

- Training-inference skew
- Error propagation
- Scheduled sampling

Production Perspective & Trade-offs

Exposure bias can cause generation models to output repetitive or irrelevant text when generating long sequences in production.

Follow-up Questions

- **Follow-up**: *How does Reinforcement Learning (RLHF) address exposure bias?* -> By optimizing the model based on the quality of the entire generated sequence, aligning training with inference behavior.

Common Mistakes

- Assuming exposure bias is a problem with the training data rather than a training-inference skew.

Question 47: Why is perplexity not always a good evaluation metric?

Short Interview Answer (30–60 seconds)

Perplexity only measures how well the model predicts the next token in the test set. It does not measure semantic correctness, factual accuracy, or logical consistency. A model can generate grammatically correct nonsense and still receive a low perplexity score.

Key Interview Points

- Next-token prediction focus
- Factual blind spots
- Tokenizer dependency

Production Perspective & Trade-offs

Perplexity is highly dependent on tokenizer vocabulary. You cannot compare perplexity scores across models that use different tokenizers.

Follow-up Questions

- **Follow-up**: *What metrics evaluate summarization quality better than perplexity?* -> BERTScore or LLM-as-a-Judge, which evaluate semantic correctness and factual consistency.

Common Mistakes

- Comparing perplexity scores between models that use different subword tokenizers.

Question 48: How does Byte Pair Encoding (BPE) differ from WordPiece and Unigram Language Models?

Short Interview Answer (30–60 seconds)

- **BPE**: A bottom-up tokenizer that iteratively merges the most frequent adjacent character pairs.
- **WordPiece**: A bottom-up tokenizer that merges pairs that maximize corpus likelihood, using a scoring ratio.
- **Unigram**: A top-down tokenizer that starts with a large vocabulary and prunes tokens that have the lowest contribution to corpus likelihood.

Key Interview Points

- Bottom-up vs. Top-down

- Frequency-based (BPE) vs. Likelihood-based (WordPiece)
- Entropy-based pruning (Unigram)

Production Perspective & Trade-offs

Unigram tokenizers offer more flexible subword segmentations, which can improve translation performance in multilingual models.

Follow-up Questions

- **Follow-up**: *Which tokenizer is used in BERT?* -> WordPiece is used in BERT, while BPE is used in GPT models.

Common Mistakes

- Assuming all subword tokenizers build vocabularies using the same merging criteria.

Question 49: Why are contextual embeddings superior to static embeddings?

Short Interview Answer (30–60 seconds)

Contextual embeddings generate dynamic vectors based on the surrounding sentence context, resolving word ambiguity (polysemy). Static embeddings assign a single fixed vector to each word, failing to handle polysemy (e.g. `"bank"` in `"river bank"` vs. `"money bank"`).

Key Interview Points

- Polysemy resolution
- Dynamic vector mapping
- Contextual semantics

Production Perspective & Trade-offs

Contextual embeddings require running transformer layers, which increases serving costs and latency compared to static embedding lookups.

Follow-up Questions

- **Follow-up**: *How does BERT construct contextual embeddings?* -> By processing tokens through multiple self-attention layers that update representations based on all other words in the sentence.

Common Mistakes

- Assuming static embeddings are obsolete (they remain useful for low-latency similarity searches).

Question 50: What are the computational complexity differences between RNNs and self-attention?

Short Interview Answer (30–60 seconds)

- **RNN**: Sequential updates scale as $O(L \cdot d^2)$ time complexity. Path length between distant tokens scales as $O(L)$ sequential steps, preventing parallel training.
- **Self-Attention**: Matrix operations scale as $O(L^2 \cdot d)$ time and memory complexity. Path length between any two tokens is $O(1)$, enabling parallel training.

Key Interview Points

- $O(L \cdot d^2)$ sequential vs. $O(L^2 \cdot d)$ parallel
- $O(L)$ path length vs. $O(1)$ path length
- Quadratic sequence bottleneck ($O(L^2)$)

Production Perspective & Trade-offs

While self-attention enables fast, parallel training, its quadratic memory cost ($O(L^2)$) makes serving long sequences resource-intensive.

Follow-up Questions

- **Follow-up**: *At what sequence length L does self-attention compute cost exceed RNNs?* -> When sequence length L exceeds the hidden dimension d ($L > d$), self-attention becomes more computationally expensive than RNNs.

Common Mistakes

- Assuming self-attention is always more computationally efficient than RNNs (it is only more efficient during training due to parallelization).